

"Deepfake Detection System"

A

Project Report

*submitted in partial fulfillment of the
requirements for the award of the degree of*

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE & ENGINEERING

by

Name

Dishant Saini

Sahil

Akshat Rajput

Palak Jain

Roll No.

R2142220485

R2142220622

R2142220209

R2142220583

under the guidance of

Dr. Bhavana Kaushik



School of Computer Science

University of Petroleum & Energy Studies

Bidholi, Via Prem Nagar, Dehradun, Uttarakhand

[Signature]
7/5/2025

[Signature]
(01/05/2025)

“Deepfake detection system”

A

Project Report

*submitted in partial fulfilment of the
requirements for the award of the degree of*

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE & ENGINEERING

by

Name	Roll No.
Dishant Saini	R2142220485
Sahil	R2142220622
Akshat Rajput	R2142220209
Palak Jain	R2142220583

under the guidance of

Dr. Bhavana Kaushik



School of Computer Science

University of Petroleum & Energy Studies

Bidholi, Via Prem Nagar, Dehradun, Uttarakhand

May – 2025

CANDIDATE’S DECLARATION

I/We hereby certify that the project work entitled “**Deepfake detection system**” in partial fulfilment of the requirements for the award of the Degree of BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE AND ENGINEERING with specialization AIML and submitted to the Department of Systemics, School of Computer Science, University of Petroleum & Energy Studies, Dehradun, is an authentic record of my/ our work carried out during a period from **Jan, 2025** to **May, 2025** under the supervision of **Dr. Bhavana Kaushik**.

The matter presented in this project has not been submitted by me/ us for the award of any other degree of this or any other University.

This is to certify that the above statement made by the candidate is correct to the best of my knowledge.

Date: 30th April, 2025
Kaushik

Dr. Bhavana

Project Guide

ACKNOWLEDGEMENT

We wish to express our deep gratitude to our guide **Dr. Bhavana Kaushik** for all advice, encouragement and constant support she has given us throughout our project work. This work would not have been possible without his support and valuable suggestions.

We sincerely thanks to our respected **Dr. Ajay Prasad , Cluster Head , School of Computer Science**, for his great support in doing our project in **Deep-fake Video Detection** .

We are also grateful to Dean SoCS UPES for giving us the necessary facilities to carry out our project work successfully. We also thanks to our Course Coordinator and Activity Coordinator Dr. Sonal Talreja for providing timely support and information during the completion of this project.

We would like to thank all our **friends** for their help and constructive criticism during our project work. Finally, we have no words to express our sincere gratitude to our **parents** who have shown us this world and for every support they have given us.

Name	Dishant	Sahil	Palak	Akshat Rajput
Roll No.	R2142220485	R2142220622	R2142220583	R2142220209

Abstract

With the rise in computational power, deep learning has become so advanced that creating highly realistic fake videos—commonly known as deepfakes—has become surprisingly easy. These face-swapped videos can be misused in harmful ways, such as spreading political misinformation, staging fake terror attacks, creating revenge porn, or blackmailing individuals.

In this project, we used “deep learning models” to find out how a real video differs from a fake one. Our method is specifically built to detect both face replacement and facial reenactment deepfakes. In essence, we're using Artificial Intelligence (AI) to fight back against the threats posed by AI.

Our system combines two powerful models: a ResNeXt-based “Convolutional Neural Network (CNN)” to examine visual features frame by frame, and a “Long Short-Term Memory (LSTM) Recurrent Neural Network (RNN)” to detect patterns over time and determine whether the video has been tampered with.

To ensure the project operates well in practical conditions, we trained and tested it using a massive, balanced data corpus that mixes several well-known deepfake datasets—“FaceForensics++”, the Deepfake Detection Challenge dataset, and Celeb-DF. Despite” using a relatively straightforward and robust approach, our model delivers competitive results in identifying fake content.

Contents

1	Synopsis	
2	Technical Keywords	2
2.1	<u>Area of Project.....</u>	<u>2</u>
3	Introduction	3-5
3.1	<u>Project Idea.....</u>	<u>3</u>
3.2	<u>Motivation of the Project.....</u>	<u>4</u>
3.3	<u>Literature Survey.....</u>	<u>5</u>
4	Problem Definition and scope	6-9
4.1	<u>Problem Statement.....</u>	<u>6</u>
4.1.1	<u>Goals and objectives.....</u>	<u>6</u>
4.1.2	<u>Statement of scope.....</u>	<u>7</u>
4.2	<u>Major Constraints.....</u>	<u>7</u>
4.3	<u>Methodologies of Problem solving.....</u>	<u>7-8</u>
4.3.1	<u>Analysis.....</u>	<u>7</u>
4.3.2	<u>Design.....</u>	<u>8</u>
4.3.3	<u>Development.....</u>	<u>8</u>
4.3.4	<u>Evaluation.....</u>	<u>8</u>
4.4	<u>Outcome.....</u>	<u>8</u>
4.5	<u>Applications.....</u>	<u>8</u>
4.6	<u>Hardware Resources Required.....</u>	<u>8</u>
4.7	<u>Software Resources Required.....</u>	<u>9</u>
5	Project Plan	10-13
5.1	<u>Project Model Analysis.....</u>	<u>10</u>
5.2	<u>Risk Management w.r.t. NP Hard analysis.....</u>	<u>10-11</u>
5.2.1	<u>Risk Identification.....</u>	<u>10</u>
5.2.2	<u>Risk Analysis.....</u>	<u>11</u>
5.3	<u>Project Schedule.....</u>	<u>12-13</u>
5.3.1	<u>Project task set.....</u>	<u>12</u>
5.3.2	<u>Timeline chart.....</u>	<u>13</u>

6	Software requirement specification	14-20
6.1	<u>Introduction.....</u>	<u>14</u>
6.1.1	<u>Purpose and Scope of Document.....</u>	<u>14</u>
6.1.2	<u>Use Case View.....</u>	<u>14</u>
6.2	<u>Functional Model and Description.....</u>	<u>15-20</u>
6.2.1	<u>Data Flow Diagram.....</u>	<u>15-16</u>
6.2.2	<u>Activity Diagram:.....</u>	<u>17-18</u>
6.2.3	<u>Non-Functional Requirements:.....</u>	<u>19</u>
6.2.4	<u>Sequence Diagram.....</u>	<u>20</u>
7	Detailed Design Document	21-28
7.1	<u>Introduction.....</u>	<u>21-22</u>
7.1.1	<u>System Architecture.....</u>	<u>21</u>
7.2	<u>Architectural Design.....</u>	<u>23-28</u>
7.2.1	<u>Module 1: Dataset Gathering.....</u>	<u>23</u>
7.2.2	<u>Module 2: Pre-processing.....</u>	<u>24</u>
7.2.3	<u>Module 3: Data-set split.....</u>	<u>25</u>
7.2.4	<u>Module 4: Model Architecture.....</u>	<u>25</u>
7.2.5	<u>Module 5: Hyper-parameter tuning.....</u>	<u>27</u>
8	Project Implementation	29-37
8.1	<u>Introduction.....</u>	<u>29</u>
8.2	<u>Tools and Technologies Used.....</u>	<u>29-30</u>
8.2.1	<u>Planning.....</u>	<u>29</u>
8.2.2	<u>UML Tools.....</u>	<u>29</u>
8.2.3	<u>Programming Languages.....</u>	<u>30</u>
8.2.4	<u>Programming Frameworks.....</u>	<u>30</u>
8.2.5	<u>IDE.....</u>	<u>30</u>
8.2.6	<u>Versioning Control.....</u>	<u>30</u>
8.2.7	<u>Platform.....</u>	<u>30</u>
8.2.8	<u>Libraries.....</u>	<u>30</u>
8.3	<u>Algorithm Details.....</u>	<u>30-37</u>
8.3.1	<u>Dataset Details.....</u>	<u>30</u>
8.3.2	<u>Preprocessing Details.....</u>	<u>31</u>

8.3.3	<u>Model Details.....</u>	31-35
8.3.4	<u>Model Training Details.....</u>	35-37
8.3.5	<u>Model Prediction Details.....</u>	37
9	Software Testing	38-39
9.1	<u>Type of Testing Used.....</u>	38
9.2	<u>Test Cases and Test Results.....</u>	39
10	Results and Discussion	40-43
10.1	<u>Screen shots.....</u>	40-43
10.2	<u>Outputs.....</u>	43
10.2.1	<u>Model results.....</u>	43
11	Deployment and Maintenance	44
11.1	<u>Deployment.....</u>	44
11.2	<u>Maintenance.....</u>	44
12	Conclusion and Future Scope	45
12.1	<u>Conclusion.....</u>	45
12.2	<u>Future Scope.....</u>	45
A	References	46
B	Project Planner	47-48

Chapter 1

Synopsis

Deepfakes are synthetic videos created using AI tactics like “Generative Adversarial Networks (GANs) and Autoencoders”. These technologies allow developers to overlay facial swap, producing extremely realistic results. The quality is often so convincing that it's hard to distinguish visually. In this work, we introduce a deep learning-based approach designed to reliably detect if a video is manipulated or genuine. Our approach takes advantage of a key weakness in the deepfake creation process: even though these videos look flawless to humans, the current tools used to generate them often leave behind subtle traces or "artifacts" in each frame. While these imperfections may go unnoticed by viewers, a well-trained neural network can detect them. We specifically use a ResNeXt-based Convolutional Neural Network (CNN) to identify these hidden patterns. Our findings show that this model can effectively pick up on the unique artifacts left by deepfake generation tools, allowing it to accurately distinguish deepfakes from authentic videos.

Our system starts by using a ResNeXt deep learning architecture to derive key characteristics from each video frame. These frame-level features are then passed to a “Long Short-Term Memory (LSTM) based Recurrent Neural Network (RNN)”, which helps us determine whether a video has been manipulated or is authentic—that is, whether it's a fake or a real video. To make our solution more effective in real-world situations, we evaluated it using a large collection of deepfake videos gathered from various online sources. Our aim was to improve its performance on real-time content, so we trained it using a mix of existing datasets. This approach helped the model learn from a diverse range of facial manipulations and video qualities. We sourced a significant number of training videos from well-known datasets like “FaceForensics++, the Deepfake Detection Challenge, and Celeb-DF”. To further test its real-world capabilities, we also evaluated the model on video data collected from YouTube. This combination of training and testing ensures that our approach remains robust and delivers strong results in practical, real-time scenarios.

Chapter 2

2.1 Area of the project

Our project falls under the domain of “Deep Learning”, a part of AI research that mimics how the human brain processes information using neural networks. A key component of our system is computer vision, which enables the analysis of video frames and visual content. For this, we rely on OpenCV to handle video and frame-level processing.

At the core of our detection system is a model trained using PyTorch. This model acts as a classifier, determining whether a given video is a genuine (pristine) recording or a deepfake.

Keywords:

- CNN
- LSTM
- GAN
- PyTorch
- OpenCV

Chapter 3

Introduction

3.1 Project Idea

Amid the boom of online platforms, deepfakes have emerged as one of the most alarming threats posed by artificial intelligence. These hyper-realistic, face-swapped videos have been misused in various harmful ways—from spreading political misinformation and violent acts, to revenge porn and blackmail. Public figures like Brad Pitt and Angelina Jolie have been portrayed in fabricate footage, emphasizing the severity of the issue. With deepfakes growing more realistic, identifying them is becoming critical. In our project, we’re using AI to combat AI. Deepfakes are typically created using tools such as FaceApp and Face Swap, which rely on pre-trained neural networks like GANs (Generative Adversarial Networks) and Autoencoders.

Our method leverages a two-part deep learning architecture: a “ResNeXt Convolutional Neural Network (CNN)” extracts visual features from each frame, and a “Long Short-Term Memory (LSTM) based Recurrent Neural Network (RNN)” examines the temporal sequence of those frames to determine whether the video is authentic or manipulated. To improve performance on real-world videos, we trained our model on a large, balanced dataset composed of samples from several publicly available sources, including FaceForensics++, the Deepfake Detection Challenge dataset, and Celeb-DF. This variety helps the model recognize different types of manipulations and generalize better to new content.

To make our solution user-friendly and easy to use, we’ve built a basic user interface. Users can upload any video through the interface, and our system will analyse it and return the result—identifying whether the video is a deepfake or real, along with a confidence score.

3.2 Motivation

With the rapid advancement of mobile camera technology and the widespread use of social media and video-sharing platforms, creating and sharing digital videos is more effortless than before. At the same time, deep learning has opened the door to tools once thought impossible just a few years ago. One such innovation is the emergence of modern generative models, capable of producing incredibly realistic images, audio, music, and even full-motion videos.

These models have been used in many positive ways—making technology more accessible through text-to-speech systems or generating synthetic training data for fields like medical imaging. However, like any powerful innovation, they also come with new challenges.

One alarming consequence has been the rise of "deepfakes"—AI-generated videos and audio clips that convincingly alter reality. Since they first appeared around 2017, deepfakes have become increasingly common, thanks to the availability of open-source tools that make the

technology accessible to the public. While some of these videos are created as jokes or for entertainment, others have the potential to do serious harm.

The growing proliferation of deepfakes on social networks has triggered the spread of false information and dangerous content. For example, imagine a deepfake showing a national leader declaring war, or a fabricated video of a well-known celebrity insulting their fans. Such content can incite panic, damage reputations, and mislead the public.

To combat this escalating issue, the need for reliable deepfake detection has become critical. In response, our project presents a deep learning-based method designed to accurately distinguish between real and AI-generated (deepfake) videos. Developing technology that can catch these manipulations is essential to preventing the spread of misinformation and protecting the integrity of digital content online.

3.3 Literature Survey

Face Warping Artifacts introduced a method that detects inconsistencies between the generated face area and its surrounding regions using a dedicated “Convolutional Neural Network (CNN)”. Their technique leverages the fact that most deepfake algorithms generate lower-resolution images that must be warped to fit the source video’s face. However, this method focuses only on spatial artifacts and lacks sensitivity to motion or time-based features.

Eye Blinking Detection proposed a novel approach where the presence or absence of eye blinking was used as an indicator to classify videos as deepfake or real. A Long-term “Recurrent Convolutional Network (LRCN)” was used to perform temporal analysis on cropped frames around the eyes. While this was effective initially, modern deepfake generators have improved significantly, and missing blinks alone are no longer a reliable detection feature. Additional cues—such as unnatural facial expressions, teeth enhancement, misplaced eyebrows, or inconsistent regular facial lines—must also be considered.

Capsule Networks for Forgery Detection applied capsule networks to detect forged content in scenarios like replay attacks and computer-generated videos. Although the model showed promising results on their dataset, it was trained with random noise, which can reduce reliability when applied to real-world data. In contrast, our approach avoids introducing noise during training and emphasizes performance on realistic, noiseless datasets.

RNN-Based Deepfake Detection employed “Recurrent Neural Networks (RNNs)” alongside ImageNet-pretrained models to analyse frame sequences. However, their training was limited to the HOHO dataset, which contains only 600 videos of similar types. Such a small, homogeneous dataset may hinder the model’s ability to generalize to varied real-time content. Our proposed system, by contrast, is trained on a significantly larger and more diverse set of real-world videos to deliver strong results and reliability in applied settings.

Synthetic Portrait Videos using Biological Signals presents a unique approach to deepfake detection by leveraging biological signals extracted from facial regions. The method compares real and deepfake portrait video pairs, analysing their spatial and temporal patterns. It evaluates vital signal patterns using feature vectors and photoplethysmography (PPG) maps—subtle color changes in the skin caused by blood flow—and uses these to train both a probabilistic Support Vector Machine (SVM) and a Convolutional Neural Network (CNN). The final

classification is based on the average of authenticity probabilities, determining whether the video is real or manipulated.

FakeCatcher is another method that stands out for its ability to detect fake videos with high accuracy, regardless of the fake video creation method applied, or the video's content, resolution, or quality. However, a major challenge with this method lies in its signal preservation strategy. Since the model lacks a discriminator component, it struggles to consistently retain biological signal integrity. Designing a differentiable loss function that aligns with its signal-processing pipeline remains a complex task.

Chapter 4

Problem Definition and Scope

4.1 Problem Statement

While convincing manipulations of digital images and videos have been possible for decades using traditional visual effects, recent breakthroughs in deep learning have taken the realism of fake content to an entirely new level. Today, AI-synthesized media—commonly known as deepfakes—can be created with remarkable ease and accuracy.

Although generating deepfakes has become relatively simple with the help of AI tools, detecting them remains a significant challenge. History already offers alarming cases where deepfakes were employed to fuel political tension, fabricate acts of terrorism, create revenge porn, and blackmail individuals. Given the speed of widespread dissemination of deepfakes across social media platforms, it is crucial to develop effective detection methods to prevent the spread of misinformation and protect public trust.

In response to this urgent need, our project takes a step forward by leveraging a Long Short-Term Memory (LSTM) based Artificial Neural Network to detect deepfake videos with greater reliability.

4.1.1 Goals and Objectives

- **Reveal the Distorted Reality Behind Deepfakes**
Our project is focused on uncovering the hidden manipulations within deepfake videos and bringing the truth to light.
- **Minimize Abuse and Misinformation**
By detecting deepfakes, our system aims to reduce the misuse of AI technology and protect people from being misled or harmed through fake content on the internet.
- **Accurately Classify Videos**
Our model is designed to reliably distinguish and classify videos as either deepfakes or authentic (pristine) videos.
- **Provide an intuitive detection interface**
We provide a simple and accessible system that allows users to upload a video and quickly receive a result showing if the video is authentic or manipulated.

4.1.2 Statement of Scope

While there are many tools available today for creating deepfakes, there are very few effective solutions for detecting them. Our approach offers a valuable contribution towards reducing the circulation of deepfakes.

We are building a web-based platform where users can easily upload a video and receive a classification—identifying whether the video is real or fake. Looking ahead, this project has strong potential for scaling. It could evolve from a standalone web application into a browser plugin capable of automatically detecting deepfakes in real time. Major platforms like WhatsApp and Facebook could even integrate this system into their applications, enabling the pre-screening of media before users share it.

Additionally, the technical specifications of our software include descriptions of input size requirements, input bounds, validation rules, input dependencies, major input and output types, and the corresponding I/O state diagrams—all outlined without diving into specific implementation details.

4.2 Major Constraints

- **User Access and Detection**
Users can upload any video to the platform and quickly find out whether the content is authentic or a deepfake. Along with the classification result, the system also displays the model's confidence score for the prediction.
- **Real-Time Prediction Display**
Once the video is uploaded, users can watch the video playback with real-time output overlaid on faces. The system highlights the detection result for each face, along with the associated confidence levels.
- **Simple and User-Friendly Interface**
We designed the platform to be straightforward and easy to navigate. Users can select videos easily through a 'Browse' button and start processing with minimal steps. By keeping the process simple, we aim to eliminate unnecessary complexity and enhance the overall user experience.
- **Cross-Platform Compatibility**
Accessibility is a key priority for us. Since the system operates as a “server-side application”, it can be accessed from any device with a web browser—whether it's a smartphone, tablet, or computer. This ensures that users across different platforms can use the service without any limitations.

4.3 Methodologies of Problem Solving

4.3.1 Analysis

To ensure the feasibility of our approach, we began by thoroughly analyzing the problem statement. After confirming that a viable solution was possible, we proceeded to the dataset collection and analysis phase.

During our initial experimentation, we explored training the model using only real or only fake videos (i.e., unbalanced training). However, this approach introduced unwanted bias into the model, leading to inaccurate predictions. After extensive testing and research, we concluded that balanced training—using a well-distributed mix of real and fake videos—was the most effective way to reduce both bias and variance in the model, resulting in significantly better accuracy.

Parameters Identified

- Eye blinking patterns
- Teeth enhancement or distortion
- Unnatural distance between eyes
- Presence or distortion of facial hair (e.g., mustaches)

- Double contours on facial features (e.g., eyes, ears, nose)
- Iris segmentation inconsistencies
- Missing or inconsistent facial wrinkles
- Erratic or varying head positions
- Unnatural face angles

4.3.2 Design

After conducting thorough research and analysis, we finalized the system architecture for our deepfake detection solution. We defined the baseline architecture of our model, carefully selecting the types of layers and determining the appropriate number of layers required for optimal performance. This architecture forms the foundation of our detection system, ensuring both efficiency and accuracy in identifying manipulated videos.

4.3.3 Development

After careful analysis, we chose **Python 3** as the programming language and **PyTorch** as the deep learning framework for building our model. PyTorch was selected due to its excellent support for GPU acceleration via CUDA, as well as its flexibility and ease of customization for complex neural networks. For large-scale training, we used **kaggle** which offered the necessary computational power and scalability to handle extensive datasets.

4.3.4 Evaluation

To evaluate the performance of our trained model, we evaluated using a wide range of real-world video data, including a large collection of **YouTube videos**. We used the **Confusion Matrix** method to assess the accuracy of the predictions, analysing the true positives, false positives, true negatives, and false negatives to gain a clear understanding of the model's reliability.

4.4 Deliverables

The outcome of our project is a well-trained deepfake detection model capable of distinguishing between real and AI-generated (deepfake) videos. This model empowers users to compare the originality of any video they upload, enhancing security and trust in digital content.

4.5 Applications

We designed a web-based application that allows users to upload videos and submit them for analysis. The system automatically processes the video and uses the trained model to classify it as either deepfake or real. This solution offers a user-friendly interface, making deepfake detection accessible to a broader audience.

4.6 Hardware Resources Required

This project requires a computer with substantial processing power, as it involves intensive image and video batch processing. The deep learning models used in our system—especially during training and prediction phases—demand high computational resources, particularly for handling large volumes of video data efficiently.

Sr. No.	Parameter	Minimum Requirement
1	Intel Xeon E5 2637	3.5 GHz
2	RAM	16 GB
3	Hard Disk	100 GB
4	Graphic card	NVIDIA GeForce GTX Titan (12 GB RAM)

4.7 Software Resources Required

- Operating System: Windows 7 or higher
- Programming Language: Python 3.x
- Frameworks:
 - PyTorch 1.4 (for deep learning model development)
- Key Libraries:
 - OpenCV (for video and frame-level processing)
 - face_recognition (for detecting and tracking facial features in video frames)

Chapter 5

Project Plan

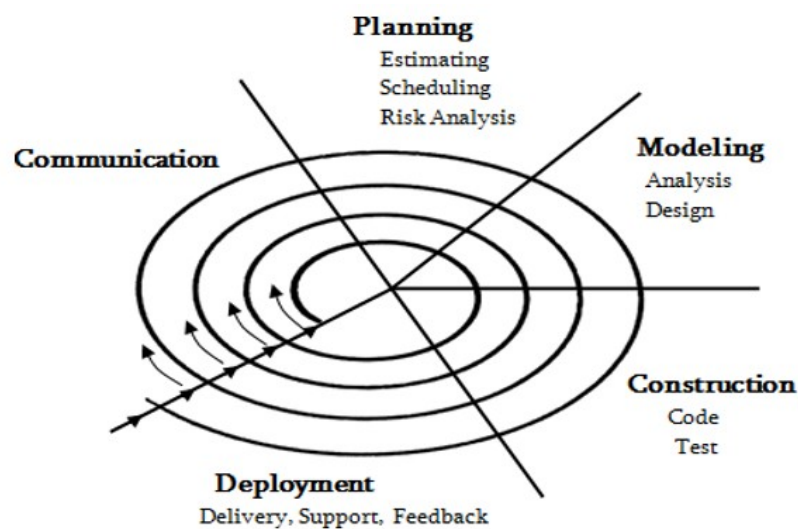
5.1 Project Model Analysis

For this project, we adopted the Spiral Model as our software development methodology. This model places strong emphasis on the people involved, their collaboration, and—most importantly—risk management throughout the development lifecycle.

The Spiral Model is especially well-suited for projects like ours, where the system is developed in multiple modules and requires ongoing adjustments. Its iterative nature allows for continuous evaluation and refinement, ensuring that changes can be made quickly and aligned with evolving requirements. Each cycle involves reviewing the current stage against expected outcomes, which helps in keeping development on track.

Another key advantage of the Spiral approach is client involvement. It provides the opportunity for stakeholders to actively participate—from feature prioritization and planning to regular review sessions. While this means clients interact with versions that may still be in progress, it provides improved clarity and flexibility during development.

Given that our project involves a high degree of technical complexity and risk, the Spiral Model's built-in mechanisms for risk identification and resolution make it an ideal choice for our product development.



5.2 Risk Handling

5.2.1 Risk Identification

Before beginning the training process, it's crucial to prepare a large number of high-quality facial images for both individuals involved in the deepfake detection model. A helpful shortcut is to use a face detection library to automatically extract facial images from video footage.

Here are some key steps to ensure optimal data preparation:

- o **Filter Out Multi-Person Frames**
Remove any video frames that contain more than one person to avoid confusion during model training.
- o **Use Diverse Footage**
Ensure you collect a wide variety of facial images showing different poses, angles, and facial expressions. Diversity in facial data helps the model generalize better to unseen videos.
- o **Focus on Facial Similarity (Optional)**
If the goal involves impersonation detection or face swaps, choosing individuals with similar facial structures may improve blending quality and realism during generation, or help spot manipulations more effectively during detection.

Spending time refining and curating your training dataset has a significant impact on the final model's accuracy and robustness.

5.2.2 Risk Analysis

In the context of deepfakes, one common issue is the appearance of **artifacts**—visual anomalies—around the edges of the swapped face, often due to mask generation used to blend the fake face with the original video.

To reduce these artifacts and improve realism:

- o **Apply** **Gaussian** **Filtering**
Use a Gaussian filter around the mask boundaries to soften the edges and blend the fake face more naturally into the original frame.
- o **Adjust** **Mask** **Size**
Modify the mask dimensions—either expanding or contracting it—based on the facial region and blending requirements.
- o **Control** **Mask** **Shape**
Fine-tune the shape of the mask to match facial contours more accurately, which helps reduce unnatural transitions and increases the visual quality of the output.

ID	Risk Description	Probability	Impact		
			Schedule	Quality	Overall
1	Does it over blur <u>comparing</u> with other non-facial areas of the video?	Low	Low	High	High
2	Does it flick?	High	Low	High	High
3	Does it have a change of skin tone near the edge of the face?	Low	High	High	Low
4	Does it have a double chin, double eyebrows, double edges on the face?	High	Low	High	Low
5	When the face is partially blocked by hands or other things, does it flick or get blurry?	High	High	High	High

5.3 Work Plan

5.3.1 Major Tasks in the Project

- **Task 1: Dataset Compilation and Evaluation**
This phase involves fetching deepfake datasets from reliable sources, analysing their structure, and preparing the data for preprocessing. The goal is to ensure the dataset is clean, diverse, and ready for the next stages.
- **Task 2: Phase 1 – Frame Extraction and Face Cropping**
In this module, input videos are broken down into frame sequences. Each frame is then processed to identify and extract only the facial region, forming the basis for a clean and focused dataset.
- **Task 3: Preprocessing**
This step focuses on building a new dataset that includes only face-cropped images and videos. These are resized, normalized, and organized for efficient model training.
- **Task 4: Phase 2 – Data Loader and Baseline Model**
A Data Loader is deployed to handle batch loading of face videos and their corresponding labels. A baseline model is initially trained using a limited portion of the dataset to validate the overall pipeline and functionality.
- **Task 5: Hyperparameter Tuning**
This stage involves adjusting key training parameters such as “learning rate, batch size, weight decay, and model architecture”. The aim is to fine-tune the system for maximum accuracy and minimal overfitting.
- **Task 6: Training the Final Model**
With the best hyperparameters identified, the model is trained on the full dataset. This step produces the final deepfake detection model.
- **Task 7: Front-End Development**
A user-friendly front-end interface is developed, allowing users to upload videos and view the classification results. This module also includes integration with the back-end model.
- **Task 8: Testing and Validation**
The complete system undergoes testing, including unit testing and functional validation, to ensure that both the model and the user interface perform reliably and as expected.

5.3.2 Timeline

Task	Timeframe	Description
Project Initiation(T0)	February 2025	Research and requirement gathering
Dataset Collection & Preprocessing (T1)	February 2025	Gathering and preprocessing deepfake datasets
Model Development (T2)	March 2025	Implementing CNN-based deepfake detection models
Testing & Optimization(T3)	March 2025	Improving model accuracy and efficiency
UI Development (T4)	April 2025	Creating a user-friendly detection interface
Final Testing & Report Submission (T5)	End Semester	Comprehensive evaluation and documentation

Chapter 6

Software Requirement Specification

6.1 Introduction to requirements

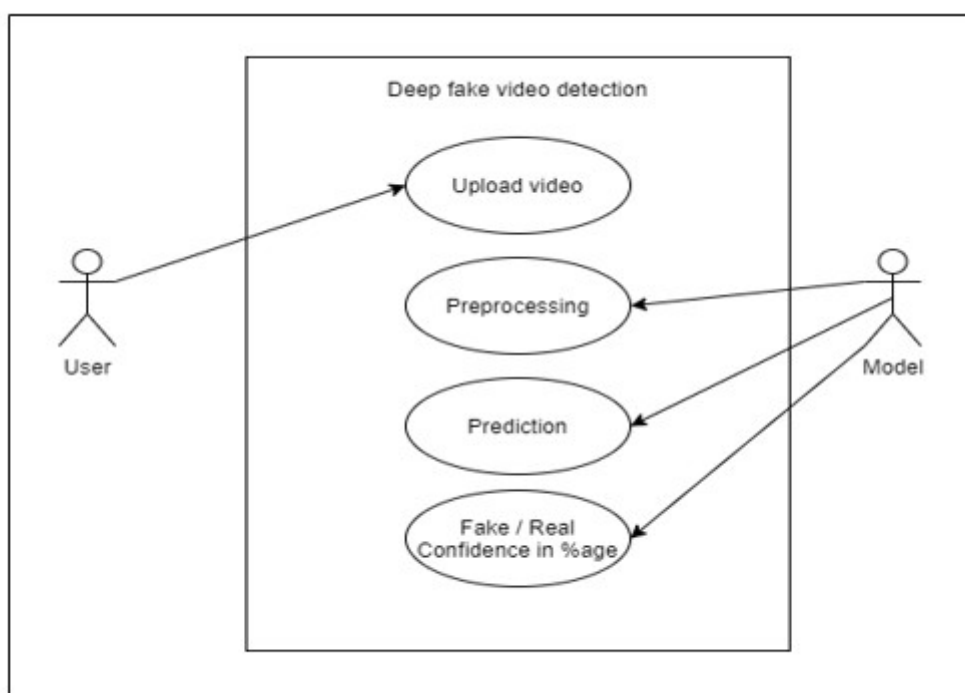
6.1.1 Objectives and coverage of the Document

This file presents the project plan to create a deepfake video detection system using neural networks. It is intended for both current and future developers working on this initiative, as well as project sponsors and stakeholders.

The document provides a thorough overview of the system and its development process. It includes—but is not limited to—a summary of system functionality, clearly stated boundaries of the project as seen by the development team, and detailed diagrams such as the “Use Case Diagram, Data Flow Diagram, and Activity Diagram.”

Additionally, the document will describe both performance and technical specifications, identify key challenges and planned responses, and present the development methodology chosen for the project. It will also outline the metrics and measurements that will be tracked throughout the development lifecycle to ensure the project remains on schedule and meets its goals.

6.1.2 Use Case View

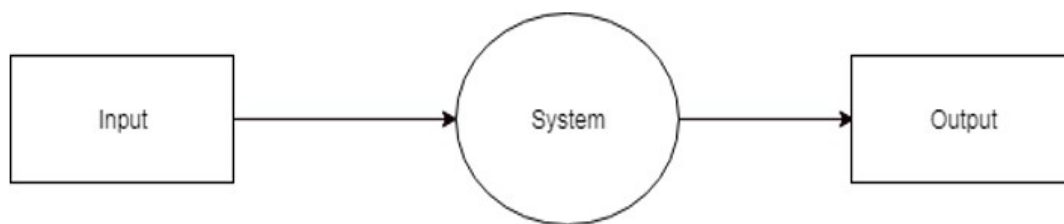


6.2 Functional Model and Description

A explanation of all key system operations is provided in this document, accompanied by supporting diagrams. For structured analysis, Data Flow Diagrams (DFDs) illustrate how data moves through the system. In the case of an object-oriented approach, an Analysis Class Diagram is presented along with descriptions of each class, outlining their attributes, behaviours, and relationships within the system architecture.

6.2.1 Data Flow Diagram

DFD level 0



The “Level 0 Data Flow Diagram (DFD)” provides a broad description of how the information moves through the system, highlighting the connection between the user, the system, and the output. It captures the basic structure of the application and places equal emphasis on both input and output processes.

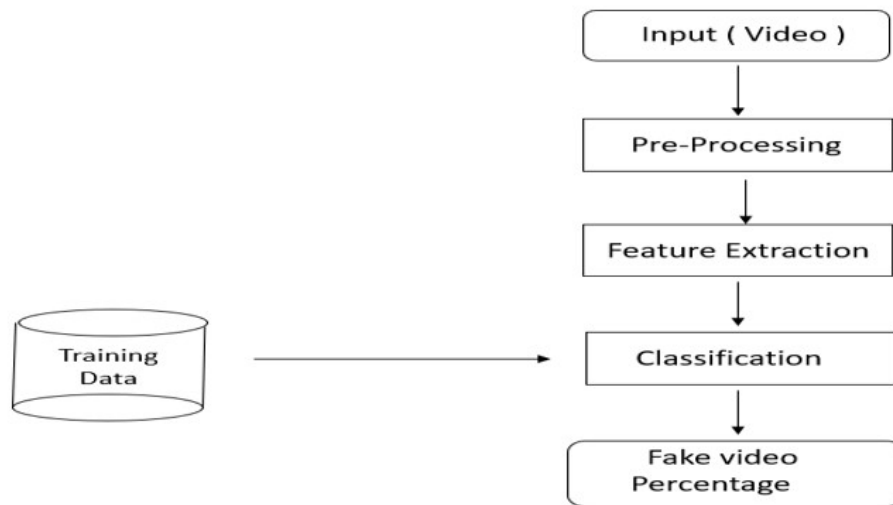
Input: The user uploads a video file to the system.

System (Process): The system processes the uploaded video, extracts relevant features, and analyses the content using the trained deepfake detection model.

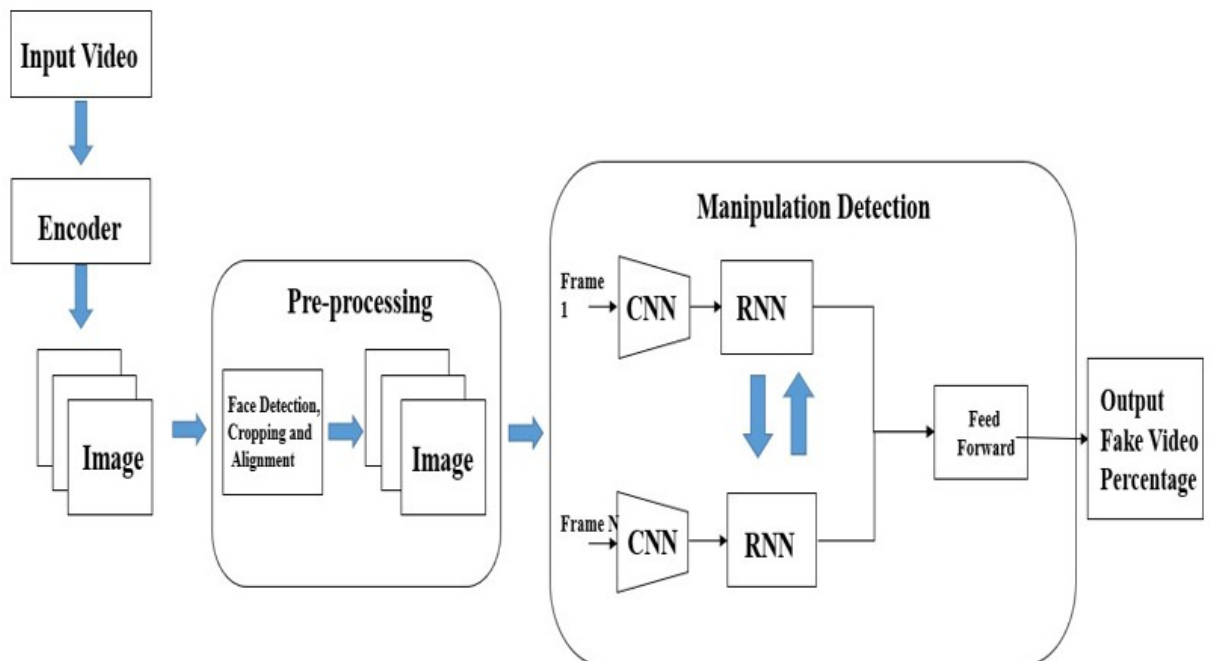
Output: The system then returns the classification result—indicating if the video is manipulated or authentic.

This DFD Level 0 diagram serves as a visual representation of the system’s core functionality, emphasizing how data enters, flows through, and exits the application.

DFD level 1

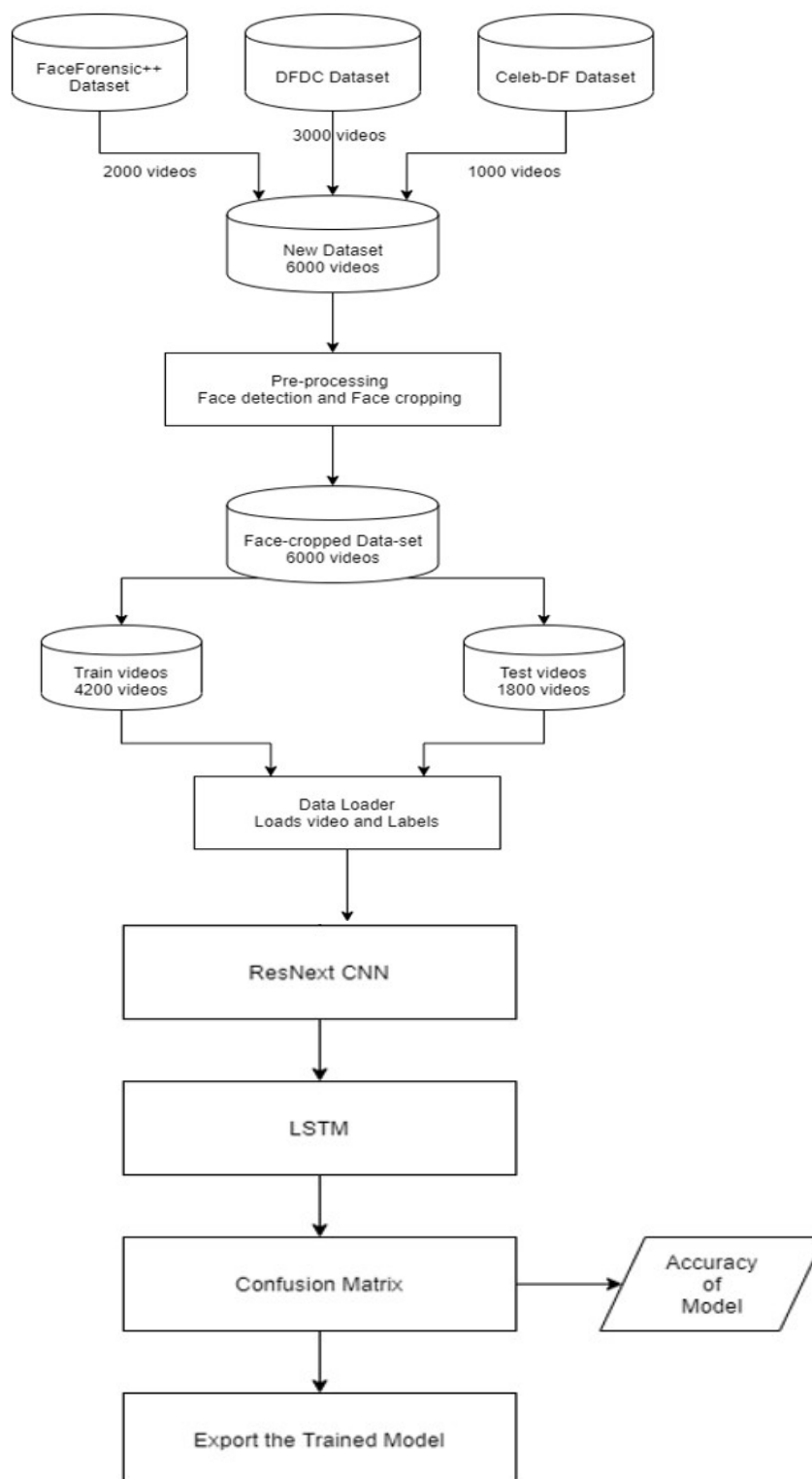


DFD level 2

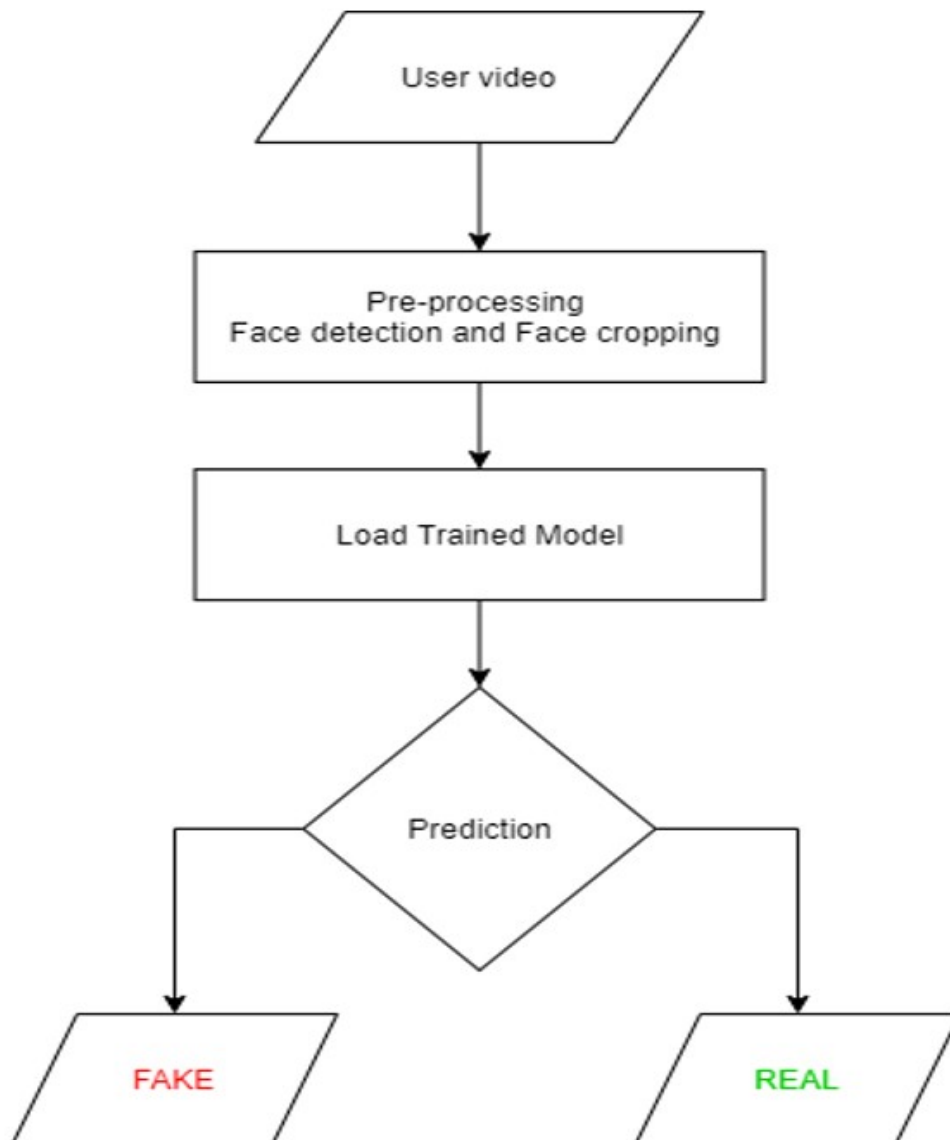


6.2.2 Activity Diagram

Training process



Testing Pipeline



6.2.3 Non-Functional Requirements

Performance Requirements

- o Efficient and Reliable Detection
The software should be designed to reliably and accurately recognize deepfake videos, ensuring it can be trusted for practical, real-world applications.
- o Versatile and User-Friendly Design
The system should offer a flexible architecture paired with an intuitive, easy-to-use interface, making it accessible to a wide range of users.

- o High Speed and Reliability
The application must deliver fast processing times without compromising on accuracy, providing users with quick and dependable results.
- o Universal Adaptability
The system should be adaptable across different platforms and devices, allowing broad accessibility without dependency on specific hardware or operating systems.
- o Future-Proof and Easy Integration
The architecture should be modular and forward compatible, making it easy to upgrade, extend, and integrate with other systems or newer technologies as needed.

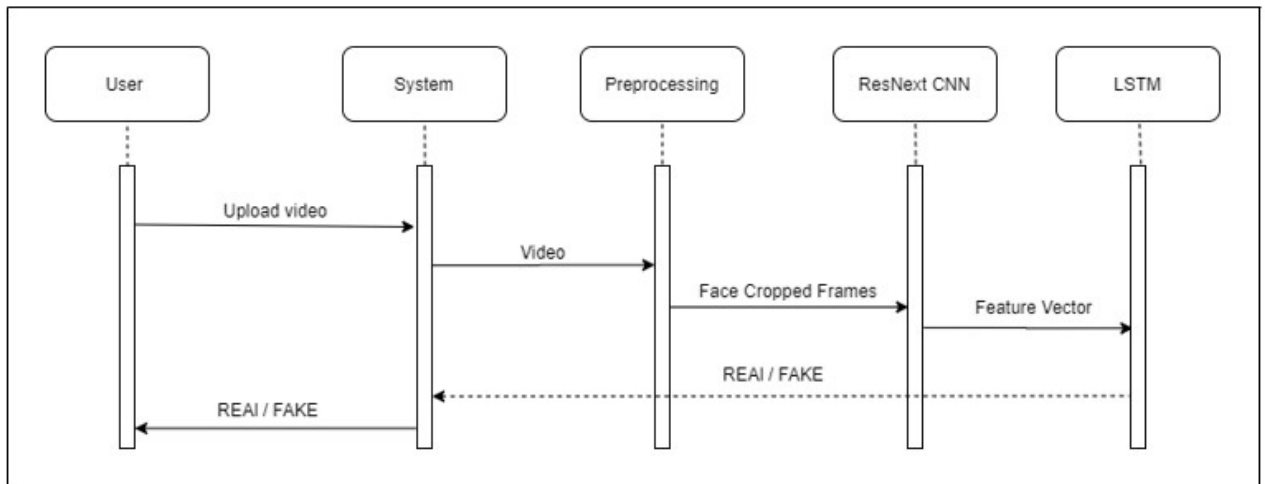
Safety Requirements

- o Data Integrity Preservation
Once a video is submitted to the system, it is directly processed by the algo without any human intervention. This ensures that the uploaded videos remain unaltered and protected from unauthorized access or manipulation.
- o Temporary Storage and Deletion Policy
To further safeguard user content, all uploaded videos are automatically deleted from the server within 30 minutes after processing. This minimizes the risk of data misuse or unauthorized retention.

Security Requirements

- o Encrypted Upload and Processing
When a user uploads a video, it is immediately encrypted using a symmetric encryption algorithm. On the server side, the video remains encrypted at rest. Decryption only occurs during the preprocessing and classification stages; once the detection process is complete, the video is re-encrypted.
- o Cryptographic Protection
This encryption process ensures the security and integrity of user data throughout the video's lifecycle within the system.
- o SSL Certification
An SSL certificate helps keep the data safe when it moves between the user's device and the server, stopping hackers from spying or changing it.

6.2.4 Sequence Diagrams

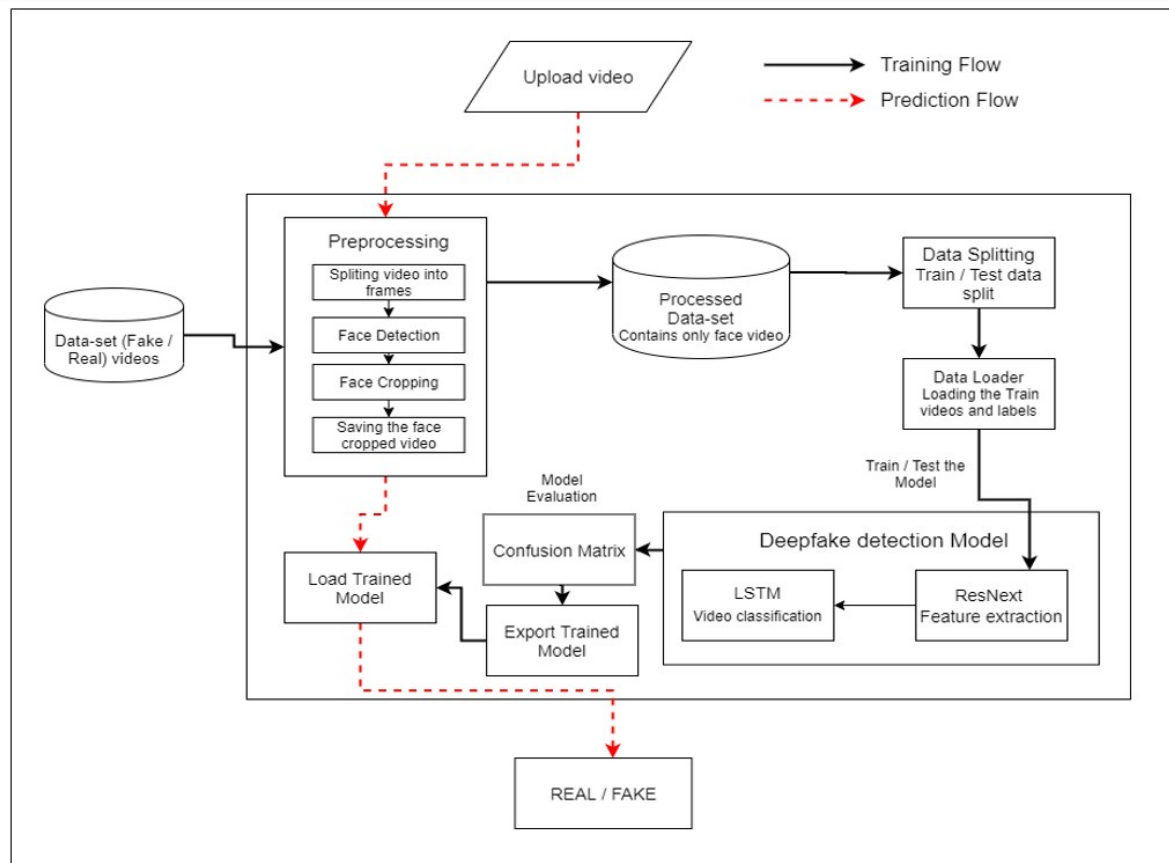


Chapter 7

Design Blueprint

7.1 Overview

7.1.1 Model framework



In this system, we trained our PyTorch-based deepfake detection model using a same count of real and fake videos to minimize skew and ensure balanced learning. The overall structural design of the model is illustrated in the accompanying figure.

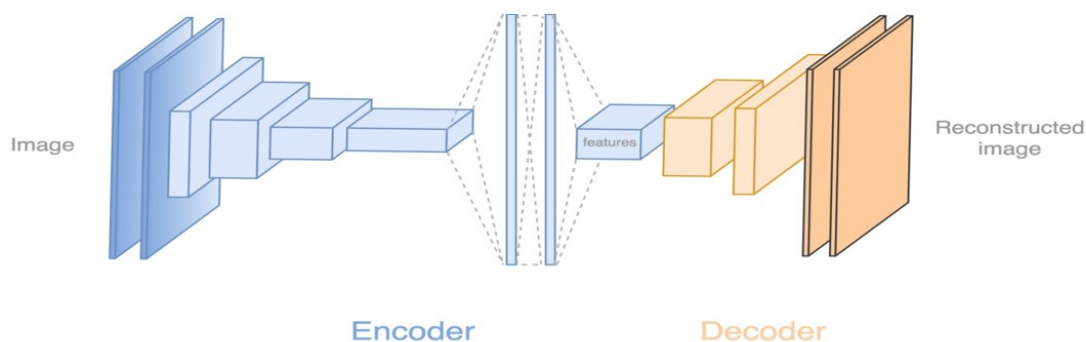
During the model creation phase, we first collected the data group, then applied preprocessing steps to refine it. As part of preprocessing, we extracted and cropped only the facial regions from the videos, creating a new, focused dataset composed exclusively of face-centered video clips. This helped in improving the model's accuracy by allowing it to concentrate on the most relevant features for deepfake detection.

Creating Deepfake Videos

To effectively detect deepfake videos, it is essential to first understand how they are created. Most deepfake generation tools—such as those based on GANs (Generative Adversarial Networks) and Autoencoders—follow a similar process. They take a source image and a target video as input, split the video into individual frames, detect the face in each frame, and replace the original face with the source face.

After face replacement, the manipulated frames are stitched back together into a video, often enhanced using pre-trained models to smooth out visible artifacts and improve the overall quality. As a result, the final deepfake video appears highly realistic, making it almost impossible to detect with the naked eye. Our project builds on this understanding to detect deepfakes. While these videos may seem flawless to human viewers, deepfake creation tools inevitably leave behind subtle artifacts or traces—small inconsistencies that are difficult to spot without computational help.

The primary objective of our work is to identify these hidden artifacts and traces using deep learning techniques, allowing us to accurately classify a video as either real or a deepfake.



Tools Commonly Used for Deepfake Creation

Several tools and frameworks are available today that make deepfake creation accessible and increasingly realistic. Some of the most widely used include:

1. **Faceswap**

An open source deepfake software that enables users to swap faces between images and videos using deep learning techniques.

2. **FaceIt**
A tool that focuses on high-quality face swapping, offering features like facial landmark alignment and real-time preview to enhance realism.
3. **DeepFaceLab**
One of the most popular deepfake creation tools, widely used by researchers and hobbyists for its flexibility and extensive control over the deepfake generation process.
4. **Deepfake Capsule GAN**
A technique that uses Capsule Networks and GANs together to create highly realistic deepfake videos with improved feature detection and facial structure preservation.
5. **Large-Resolution Face Masked Models**
Advanced models designed to handle high-resolution face swaps while maintaining fine facial details, leading to more convincing and seamless deepfake outputs.

7.2 Design Structure

7.2.1 Module 1: Dataset Compilation

To optimize our model for detecting deepfakes in real time, we collected information from several open-source datasets, namely “FaceForensics++ (FF), the Deepfake Detection Challenge (DFDC), and Celeb-DF”. We combined and curated these datasets to create a new, balanced dataset suitable for training a model capable of accurately handling a wide variety of video types.

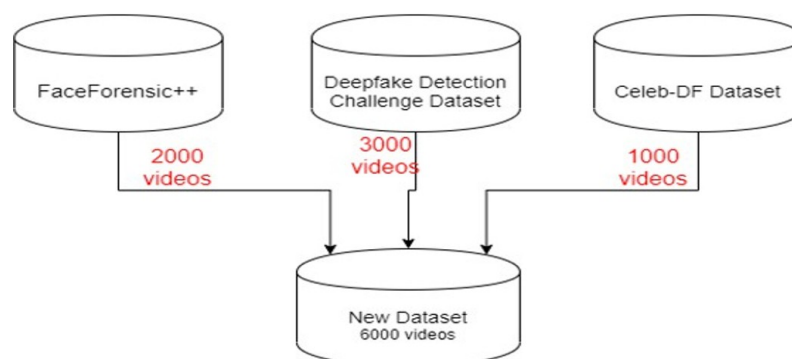
To ensure fairness and avoid model bias, we maintained a 50% real and 50% fake distribution throughout our dataset.

While working with the DFDC dataset, we identified some videos with altered audio tracks. Since detecting audio deepfakes is outside the scope of this project, we developed a Python script to automatically filter out and remove such audio-modified videos during preprocessing.

After preprocessing:

- o From the DFDC dataset: “1500 real and 1500 fake” videos were selected.
- o From the FaceForensics++ dataset: “1000 real and 1000 fake” videos were selected.
- o From the Celeb-DF dataset: “500 real and 500” fake videos were selected.

This resulted in a final dataset of 6000 videos, evenly split between 3000 real and 3000 fake samples.



7.2.2 Module 2: preprocessing

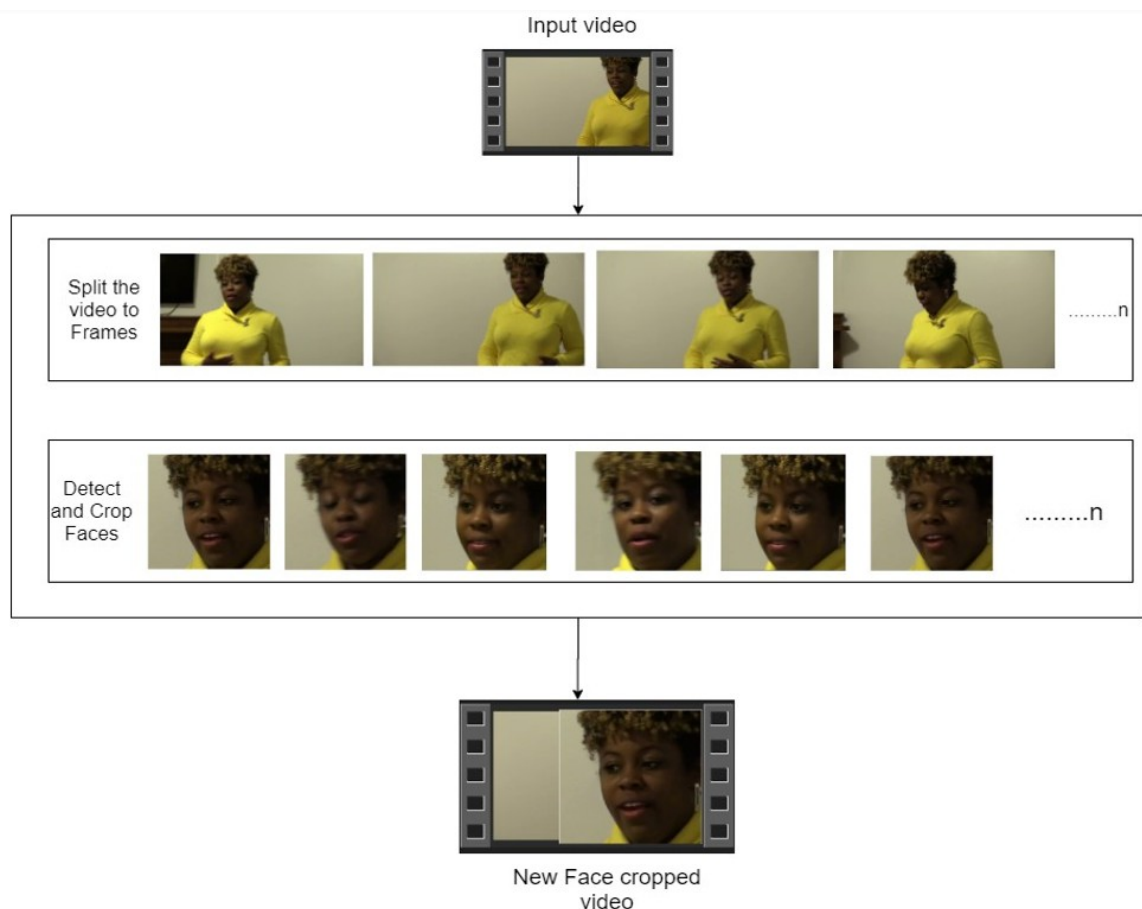
In this step, all videos undergo preprocessing to remove noise and unnecessary content, focusing only on the facial region. The first phase of preprocessing involves splitting each video into individual frames. Once the videos are segmented into frame sequences, a face identification logic is executed to each frame. If a face is detected, the frame is trimmed to isolate the facial region. Frames lacking visible faces are removed. After cropping, the facial frames are reassembled into new face-only videos, creating a clean and focused dataset optimized for training.

To maintain uniformity across all samples, we set a threshold for the number of frames based on the average number of the frames across the dataset. This was necessary for two main reasons:

- o Ensuring consistent video length for model input.
- o Managing computational resources, since processing long videos (e.g., 300 frames for a 10-second video at 30 fps) would be too resource-intensive in our experimental setup.

Given our GPU's computational limitations, we selected 150 frames as the threshold value. During preprocessing, only the first 150 sequential frames of each video were retained to preserve the natural temporal flow—important for properly leveraging the capabilities of the “Long Short-Term Memory (LSTM)” network.

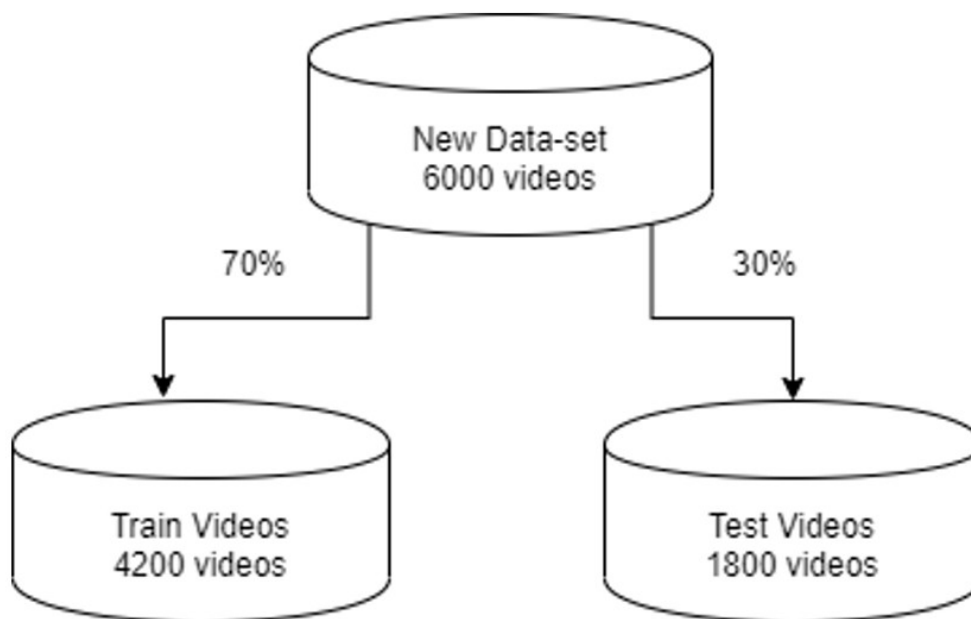
The final videos were stored at a frame rate of 30 fps and a resolution of 112×112 pixels, ensuring both efficiency and consistency across the dataset.



7.2.3 Module 3: Dataset split

The complete corpus was split into training and testing sets with a “70:30 split”. Specifically, 4,200 videos were allocated for training and 1,800 videos for testing.

To maintain consistency and avoid bias, both sets were same in terms of count, containing an equal number of real and fake videos—50% real and 50% fake in each split. This balanced distribution ensures that the model learns equally from both categories and performs accurately during evaluation.



7.2.4 Module 4: Model Architecture

The model is a combination of “Convolutional Neural Networks (CNN) and Recurrent Neural Networks (RNN)”. We use a pre-trained “ResNeXt CNN” model to extract frame-level features, and then a “Long Short-Term Memory (LSTM)” network processes these sequential features to classify the video as either a authentic or manipulated. During training, video frames and their corresponding labels are loaded using a Data Loader on the training split, and the extracted features are fitted into the model for supervised learning.

ResNeXt for Feature Extraction

Instead of building a CNN from scratch, we use the pre-trained ResNeXt model, specifically the ResNeXt-50 (32x4d) architecture, optimized for high performance in deep networks. ResNeXt is a variant of the ResNet family that improves accuracy through aggregated transformations while keeping computational complexity manageable.

In our setup:

We use a ResNeXt network with 50 layers and a 32x4d configuration.

We optimize the network by incorporating extra layers and adjusting the learning rate to ensure proper convergence during gradient descent.

The “2048-dimensional feature vectors obtained after the final pooling layer of ResNeXt are used as input to the LSTM” network for sequential processing.

LSTM for Temporal Sequence Processing

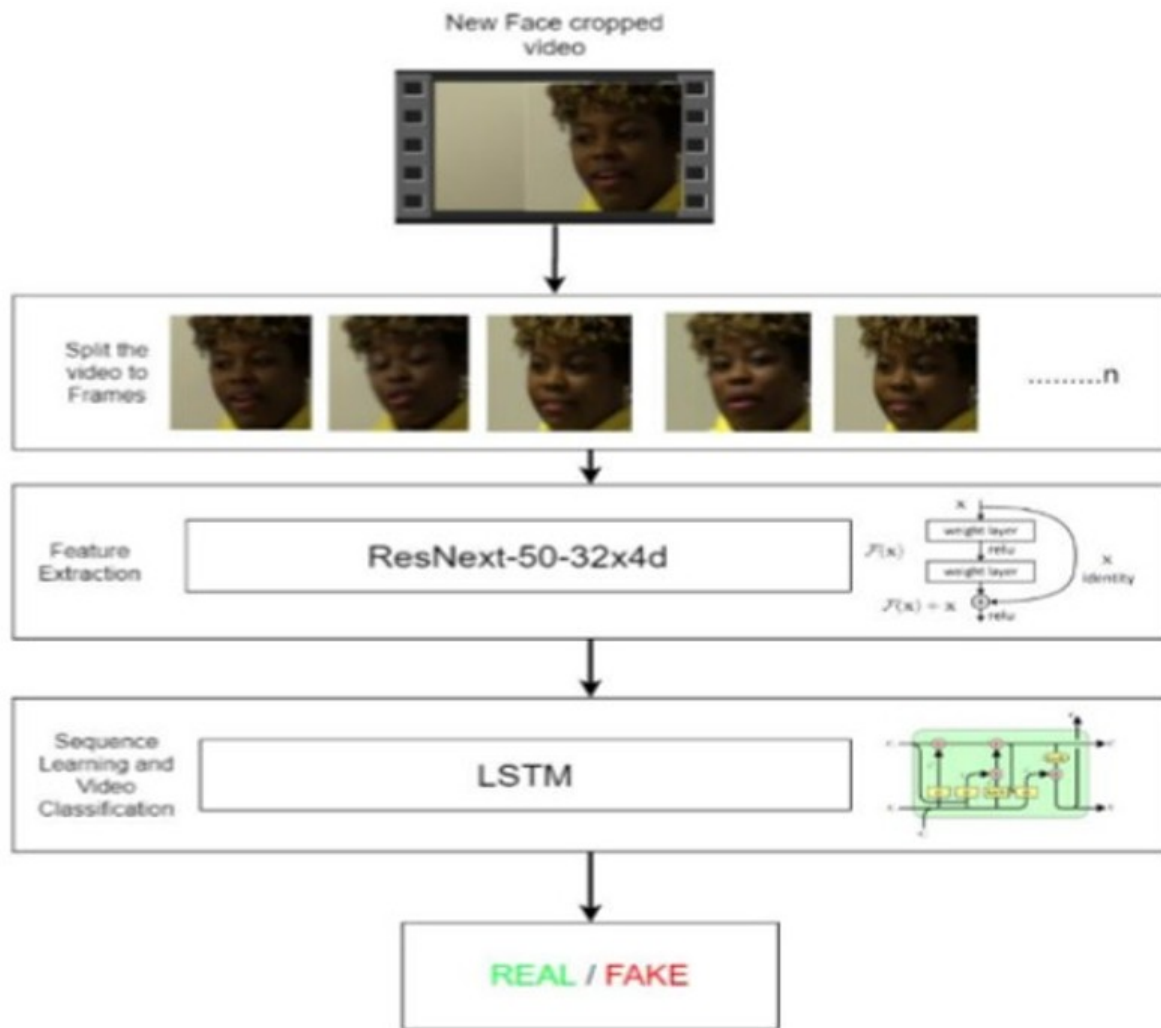
The “2048-dimensional feature vectors extracted from ResNeXt are fed into an LSTM” network designed to capture the temporal dependencies across video frames.

Key components of our LSTM setup:

- o “LSTM layer with 2048 latent dimensions and 2048 hidden units.”
- o “A dropout rate of 0.4 to prevent overfitting during training.”
- o “The LSTM processes frames sequentially, allowing temporal analysis by comparing the frame at time 't' with earlier frames like 't-n', where n can be any number of frames prior.”

Additional layers and components include:

- o Leaky ReLU activation functions for better handling of negative inputs.
- o A linear layer that maps the 2048 input features down to 2 output classes (deepfake or real).
- o Adaptive Average Pooling (with output size = 1) to dynamically reduce the feature map dimensions without losing important information.
- o A Sequential Layer to maintain proper sequence flow during batch training.
- o Batch size of 4 for efficient and manageable batch processing.
- o A SoftMax unit to find the network’s outcome probabilities and confidence scores during inference.



7.2.5 Module 5: Hyperparameter Tuning

Hyperparameter optimization is a vital phase in reaching peak model performance. After several rounds of model tuning and iterative testing, we identified the most effective set of hyperparameters for our dataset. To enable dynamic adjustment of the learning rate during training, we used the Adam optimizer, which is well-adapted for managing scattered gradients and noisy data samples. The step size was carefully adjusted to $1e-5$ (0.00001), helping the model to effectively reach convergence towards a superior global solution during loss minimization. A regularization term of $1e-3$ was also applied to mitigate overfitting.

Since this is a binary classification task (deepfake vs. real video), we applied the cross-entropy loss to calculate the model's prediction error during training.

To maximize the use of available computing capacity, we performed batch training with a batch size of 4. Based on our experiments, using 4 samples per batch was determined to offer the ideal trade-off between RAM efficiency and training throughput in our development setup.

User Interface Development

The user interface (UI) for the application was built using the Django framework, which offers scalability and flexibility for future upgrades and expansions.

The index.html page provides users with a simple and intuitive option to browse and upload a video for analysis.

Once uploaded, the video is processed by the deepfake detection model.

The model then predicts whether the video is genuine or fake, along with a confidence score.

These results are dynamically rendered onto predict.html, where the output is shown directly over the playing video for a seamless user experience.

Chapter 8

Project Implementation

8.1 Introduction

There have been numerous instances where deepfake technology has been misused to spread misinformation and mislead the public on social media. Examples include fabricated videos of prominent figures such as Mark Zuckerberg during the House A.I. Hearing, Donald Trump portrayed as James McGill from Breaking Bad, and Barack Obama delivering a fake public service announcement. These types of deepfakes can cause widespread panic and confusion, highlighting the urgent need to accurately detect and distinguish deepfakes from genuine videos.

Recent technological advances have revolutionized the field of video manipulation. The rise of modern open-source deep learning frameworks like TensorFlow, Keras, and PyTorch, combined with the easy and affordable access to powerful computational resources, has led to a dramatic shift in how media content can be tampered with.

Traditional autoencoders and pre-trained Generative Adversarial Networks (GANs) have made it significantly easier to alter realistic images and videos. Moreover, the availability of user-friendly smartphone and desktop applications like FaceApp and Face Swap has made the creation of deepfakes accessible to almost anyone. These apps allow users to transform faces in real videos with remarkable realism and offer additional features such as changing hairstyles, gender, age, and facial attributes.

Although most current uses of deepfake technology have been relatively limited, there is growing concern. Many released tools have been extensively used to create fake celebrity pornographic videos or revenge porn—targeting famous personalities like Brad Pitt and Angelina Jolie. The highly realistic appearance of these manipulated videos makes celebrities particularly vulnerable to fake surveillance footage, false news stories, and damaging hoaxes.

Deepfakes have also been increasingly weaponized for political manipulation, further escalating the risks they pose to society. Given the rapid spread of deepfakes through social media, developing effective detection technologies has become critical to prevent the percolation of fake content and protect public trust.

8.2 Tools and Technology used

8.2.1 Planning

- **OpenProject:** Used for project management and task tracking.

8.2.2 UML Tools

- **draw.io:** Used for creating UML diagrams such as Use Case, Data Flow, and Activity diagrams.

8.2.3 Programming Languages

- **Python 3:** Core language for model development and backend programming.
- **JavaScript:** Used for enhancing the web application's interactivity.

8.2.4 Programming Frameworks

- **PyTorch:** Deep learning framework used for building and training the detection model.
- **Django:** Web framework used for developing the front-end and back-end integration.

8.2.5 Integrated Development Environments (IDEs)

- **Google Colab:** Cloud-based IDE used for initial model training and experimentation.
- **Jupyter Notebook:** Used for local experiments, prototyping, and visualization.
- **Visual Studio Code:** Main code editor for web application development and model deployment scripts.

8.2.6 Version Control

- **Git:** Used for source code management and version control.

8.2.7 Platform

- **Kaggle:** Used for training the model on large datasets and hosting services.

8.2.8 Libraries and Packages

- **torch:** Core library for building neural networks in PyTorch.
- **torchvision:** Image processing utilities and pre-trained models.
- **os:** File and directory management.
- **numpy:** Numerical computations.
- **cv2 (OpenCV):** Image and video processing.
- **matplotlib:** Data visualization and plotting.
- **face_recognition:** Face detection and recognition library.
- **json:** Data serialization and handling JSON files.
- **pandas:** Data manipulation and analysis.
- **copy:** Object copying operations.
- **glob:** File pattern matching and retrieval.
- **random:** Generating random numbers and selections.
- **sklearn (scikit-learn):** Machine learning utilities and evaluation metrics.

8.3 Algorithm Details

8.3.1 Dataset Details

Refer 7.2.1

8.3.2 Preprocessing Details

The following steps were performed during the preprocessing phase:

- o Importing Videos:
All videos from the directory were imported into a Python list using the glob library.

- o **Reading Videos:**
The `cv2.VideoCapture` function from OpenCV was used to read video files and calculate the mean number of frames across the dataset.
- o **Frame Uniformity:**
To ensure consistency, 150 frames were selected as the standard for all videos, based on the computed mean frame count and considering computational constraints.
- o **Splitting and Cropping Frames:**
Each video was split into individual frames. Faces were detected within each frame, and the frames were cropped around the detected facial regions.
- o **Reconstructing Videos:**
Cropped frames were compiled into new videos using `cv2.VideoWriter`, maintaining a frame rate of 30 fps and a resolution of 112×112 pixels in MP4 format.
- o **Sequential Frame Selection:**
To properly leverage the temporal analysis capability of the LSTM, the first 150 sequential frames (rather than random frames) were selected for each new video.

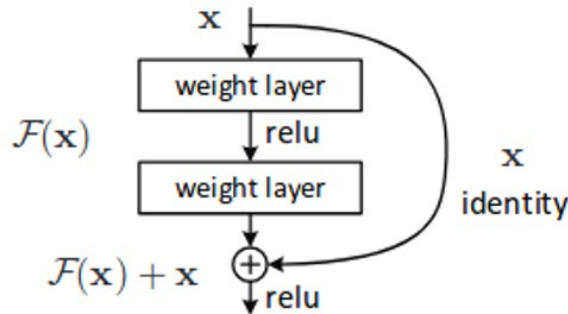
8.3.3 Model Details

The design of the deepfake detection model consists of the following components:

ResNeXt CNN (Feature Extraction Layer):

- o A pre-trained ResNeXt-50_32x4d model [22] is used to extract high-level frame features.
- o It consists of 50 layers with a $32 \times 4d$ configuration.
- o ResNeXt enhances traditional Residual Networks by using aggregated transformations for better performance and scalability.
- o The extracted feature vectors from ResNeXt (of 2048 dimensions) serve as input to the LSTM layer for sequence learning.

stage	output	ResNeXt-50 (32×4d)
conv1	112×112	7×7, 64, stride 2
conv2	56×56	3×3 max pool, stride 2
		$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128, C=32 \\ 1 \times 1, 256 \end{bmatrix} \times 3$
conv3	28×28	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256, C=32 \\ 1 \times 1, 512 \end{bmatrix} \times 4$
conv4	14×14	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512, C=32 \\ 1 \times 1, 1024 \end{bmatrix} \times 6$
conv5	7×7	$\begin{bmatrix} 1 \times 1, 1024 \\ 3 \times 3, 1024, C=32 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$
	1×1	global average pool 1000-d fc, softmax
# params.		25.0 ×10 ⁶



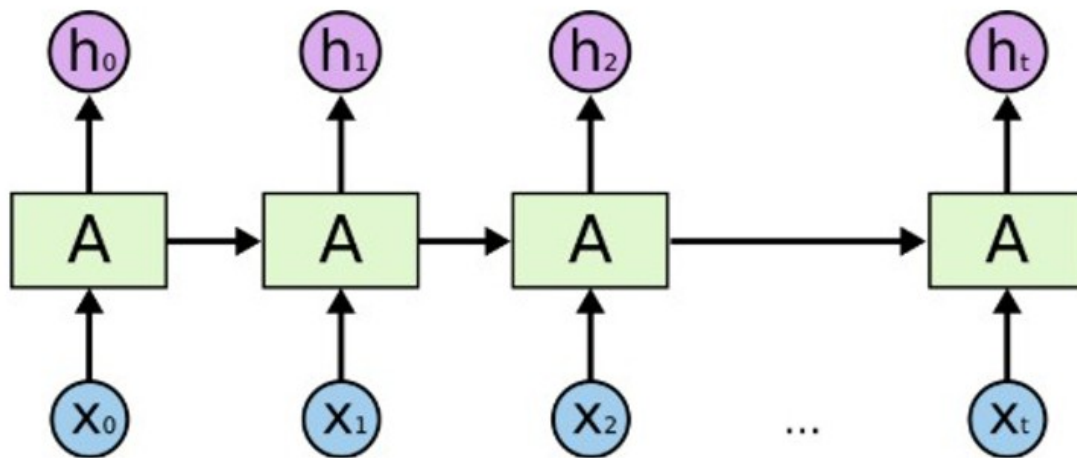
Sequential Layer

The Sequential layer acts as a container to organize multiple modules in a defined order. In our model, it is used to store and maintain the sequence of feature vectors generated by the ResNeXt model. This implies that the obtained frame-level features are processed sequentially to the LSTM layer, keeping the temporal structure necessary for detecting deepfakes.

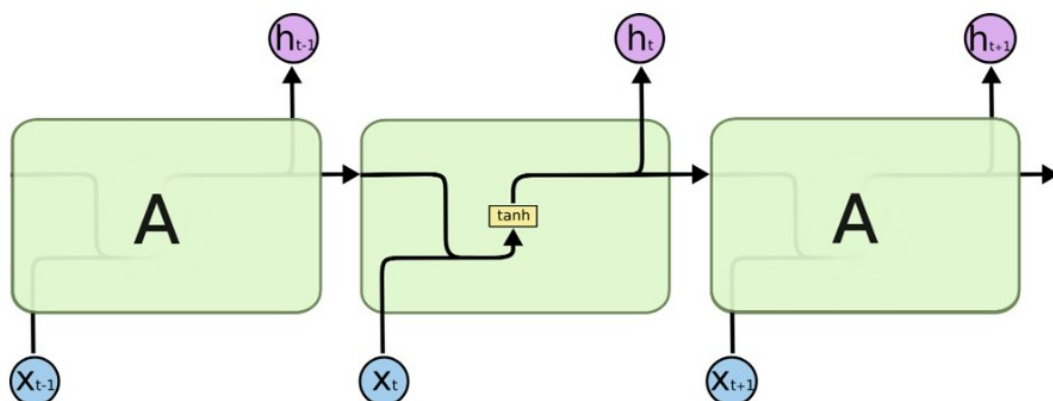
LSTM Layer

The Long Short-Term Memory (LSTM) layer is used for sequential processing and identifying temporal changes between frames. The 2048-dimensional feature vectors extracted from ResNeXt serve as the input to the LSTM. We use one LSTM layer with 2048 latent dimensions and 2048 hidden units. A dropout rate of 0.4 is applied to prevent overfitting during training. The LSTM processes frames in order, comparing each at time 't' with earlier at 't-n' seconds,

where n can vary. This temporal comparison allows the model to detect subtle inconsistencies and unnatural transitions that are often indicative of deepfake manipulation.



Overview of LSTM architecture



Internal LSTM architecture

ReLU

(Rectified Linear Unit) is an activation function commonly used in deep learning models. Its operation is straightforward:

- When the input is below 0, the output is 0.
- When the input exceeds 0, the output matches the input itself.

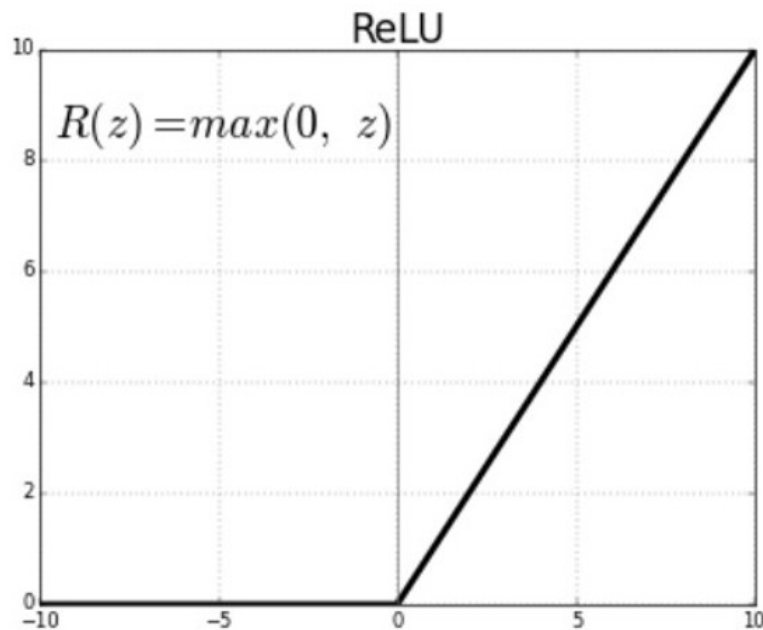
In mathematical terms, ReLU is defined as:

$$\text{ReLU}(x) = \max(0, x)$$

ReLU is closer to how biological neurons operate, making it more intuitive for modeling neural activity. It is a non-linear activation function, which allows complex patterns to be learned without introducing non-convexities that make optimization difficult.

Compared to traditional activation functions like the sigmoid, ReLU offers several advantages:

- o Faster computation: ReLU allows faster training of deep neural networks because its simple thresholding operation speeds up forward and backward passes.
- o No vanishing gradient issue: Unlike the sigmoid or tanh functions, ReLU does not suffer from vanishing gradients for positive input values, improving the effectiveness of backpropagation in deep models.
- o Due to these benefits, ReLU has become the default choice for activation functions in most modern deep learning architectures.

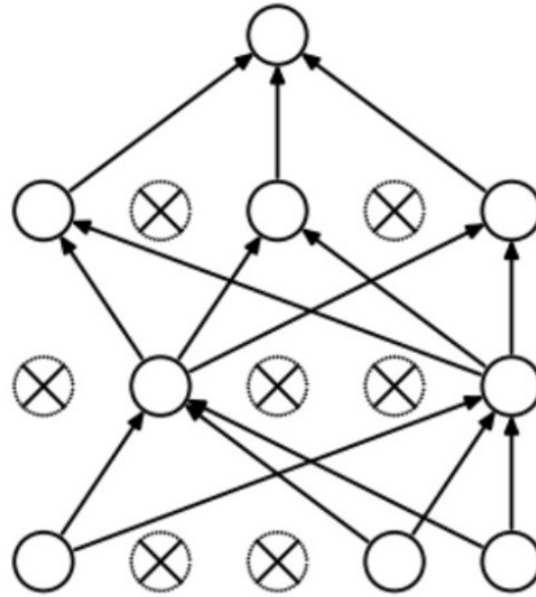


Dropout layer

Regularization technique used to prevent overfitting in deep learning models. In our model, we apply a dropout rate of 0.4, meaning 40% of the neurons are randomly "deactivated" (i.e., their output is set to zero) through each forward propagation step in training. By setting a random subset of neuron outputs to zero, the model is forced to learn more robust features that are not overly dependent on any one neuron. This encourages the network to develop redundancy and generalize better to unseen data.

When the output of certain neurons is suppressed:

- o The cost function becomes more sensitive to changes in neighbouring neurons.
- o This sensitivity alters how weights are updated during backpropagation, helping to distribute learning more evenly across the network.
- o As a result, the Dropout layer plays an important role in improving model's ability to perform well on real-world, noisy data, rather than just memorizing the training set.



Adaptive Mean Pooling Layer

The “Adaptive Average Pooling layer” is used within our system to help reduce variance, lower computational load, and extract important shallow-level attributes out of neighboring regions in the feature maps. Specifically, we use a 2-dimensional Adaptive Average Pooling layer, which dynamically adjusts the size of the output regardless of the input dimensions. This ensures that the output feature map has a fixed size, making it easier to connect the convolutional layers to fully connected layers later in the network.

By averaging the values within local neighbourhoods, this layer:

- o Smooths out small variations,
- o Highlights key structural features, and
- o Minimizes computational load during further processing stages.
- o This pooling step is particularly important for ensuring that features extracted from frames of varying sizes remain consistent for sequential analysis in the LSTM layer.

8.3.4 Model training details

Train-Test Split

- o The data collected was divided into training and testing sets with a 70:30 ratio, consisting of 4,200 videos for training and 1,800 videos for testing.
- o Each split was balanced, containing half real and half fake videos to avoid bias in model training and evaluation.

Data Loader

- o A Data Loader was implemented to efficiently load videos and their corresponding annotations during training. The videos are processed in batches of 4, helping manage computational resources and improving training stability.

Training Configuration

- o Iterations: The model was trained over 20 epochs.
- o Learning Rate: Set at $1e-5$ (0.00001) to ensure consistent convergence.
- o Weight Decay: A weight decay of $1e-3$ (0.001) was used to prevent overfitting.
- o Optimizer: Adam Optimizer was chosen for its adaptive learning rate capabilities, helping the model adjust quickly during training.

Adam Optimizer

- o The Adam optimizer merges the advantages of two other extensions of stochastic gradient descent—adaptive gradient algorithm (AdaGrad) and root mean square propagation (RMSProp). It adjusts the learning rate for each parameter dynamically, improving the productivity of gradient descent and accelerating convergence.

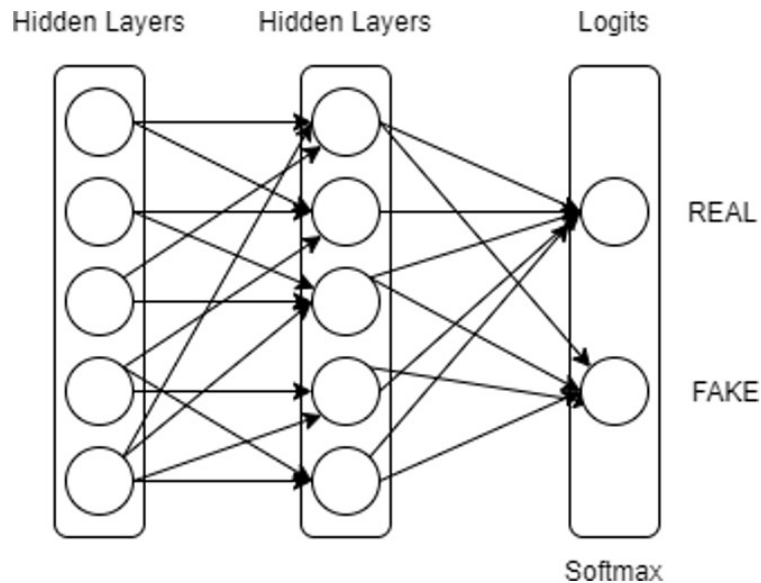
Loss Function – Cross Entropy

- o Since the task is a binary classification problem (deepfake vs. real), the Cross Entropy cost function was used to calculate the gap between the predicted probabilities and the actual labels during training.

Softmax Layer

- o The ending layer of the model is a Softmax layer, which:
- o Squashes the outputs into a range between 0 and 1.
- o Normalizes the outputs so that they sum to 1, allowing them to be interpreted as probabilities.
- o Provides confidence scores for each prediction.

In our model, the SoftMax layer has two result nodes, corresponding to the two classes: REAL or FAKE. This enables not just the classification but also the certainty degree of each prediction.



Confusion

Matrix

A “confusion matrix” is used to summarize the effectiveness of a classification model. It provides a breakdown of the correct and incorrect predictions, separated by each class (real and fake videos in our case).

The confusion matrix offers critical insights:

- It shows how and where the model is making errors.
- It highlights not just the overall accuracy but also the type of misclassifications (e.g., real videos wrongly classified as fake, and vice versa).
- It helps identify imbalances or weaknesses in model predictions that simple accuracy scores may not reveal.

In our project, the confusion matrix was used to evaluate model performance and calculate the final accuracy on the test set.

Exporting

the

Model

After successful training and evaluation, the final trained model was transfer. This gives us to easily ship the model and use for practical cases on new video uploads in the application.

8.3.5 Model Prediction Details

Model Loading

- The trained face swapped detection model is transfered into the application during runtime, ready to perform real-time predictions.

Preprocessing New Videos

- Whenever there an unseen video being uploaded for prediction, it undergoes preprocessing. This ensures consistency between the training data and the new input data.

Prediction and Output

- The pre-processed video frames are passed through the loaded model, which performs the classification.

The model then returns:

- Whether the video is classified as “REAL or FAKE.”
- The reliability score (probability) associated with the prediction, providing additional insight into the model’s certainty.

Chapter 9

Software Testing

9.1 Testing approaches applied

Functional Testing

Unit Testing

- Individual components of the application (e.g., preprocessing functions, model inference logic) are tested in isolation to ensure each unit functions correctly.

Integration Testing

- The interactions between modules—such as video upload, preprocessing, and model prediction—are tested to ensure smooth data flow and component interoperability.

System Testing

- The entire system is tested end-to-end to verify that the application behaves as expected under real-world conditions.

Interface Testing

- Testing is conducted on the user interface to ensure that user interactions (e.g., uploading a video, receiving predictions) function correctly and deliver accurate feedback.

Non-Functional Testing

Performance Testing

- o Evaluates the system's responsiveness and stability under normal operation, particularly the time taken for prediction and result rendering.

Load Testing

- o Assesses how the system handles multiple simultaneous video uploads and predictions to ensure reliability under heavier traffic.

Compatibility Testing

- o Ensures the application works smoothly across various platforms, browsers, and devices, given its web-based nature.

9.2 Test Cases and Results

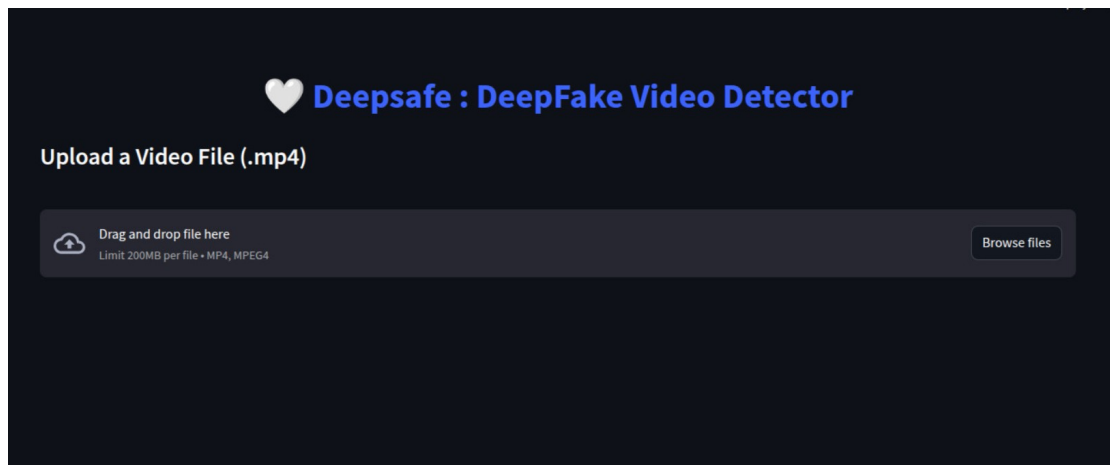
Test Cases

Case id	Test Case Description	Expected Result	Actual Result	Status
1	Upload a Real video	Real	Real	Pass
2	Videos with many faces	Fake / Real	Fake	Pass
3	Upload a face cropped real video	Real	Real	Pass
4	Deepfake video	Fake	Fake	Pass

Chapter 10

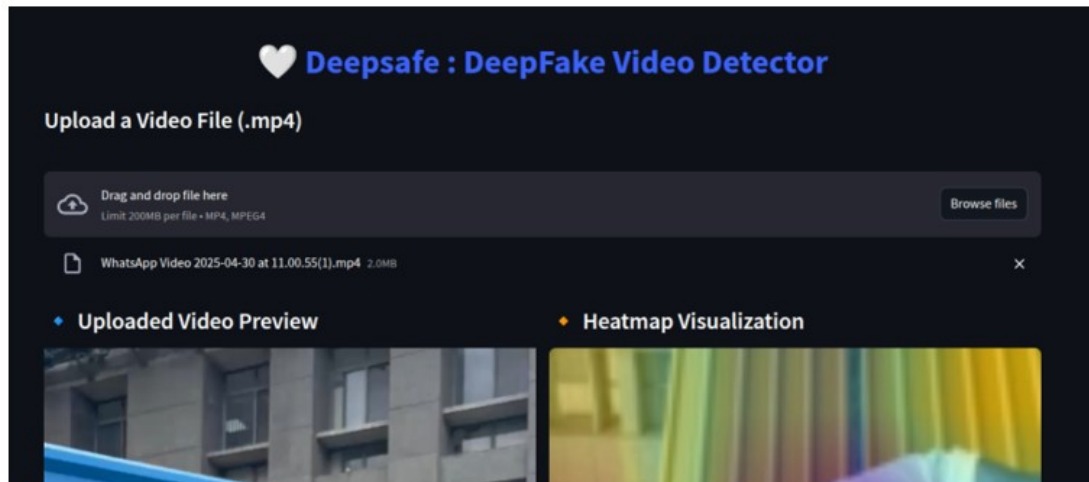
Outputs and Discussions

10.1 Screenshots

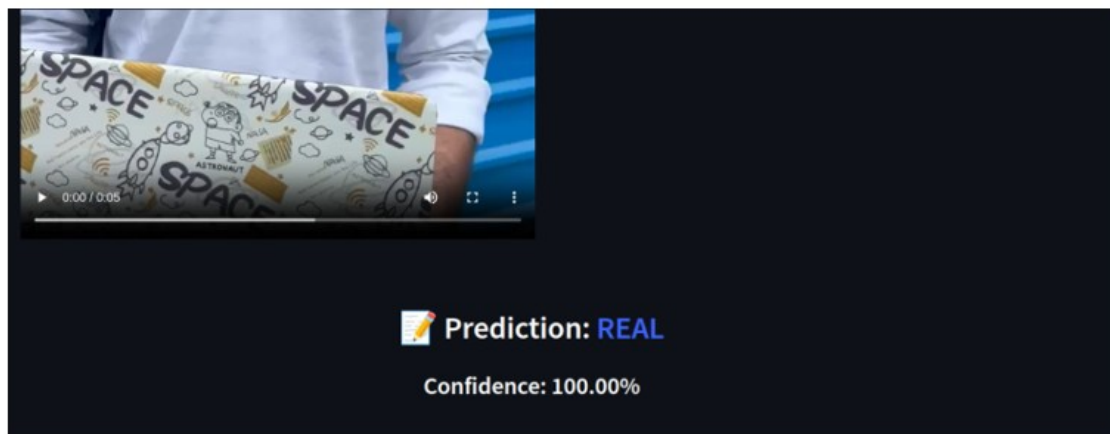


Home Page

1.

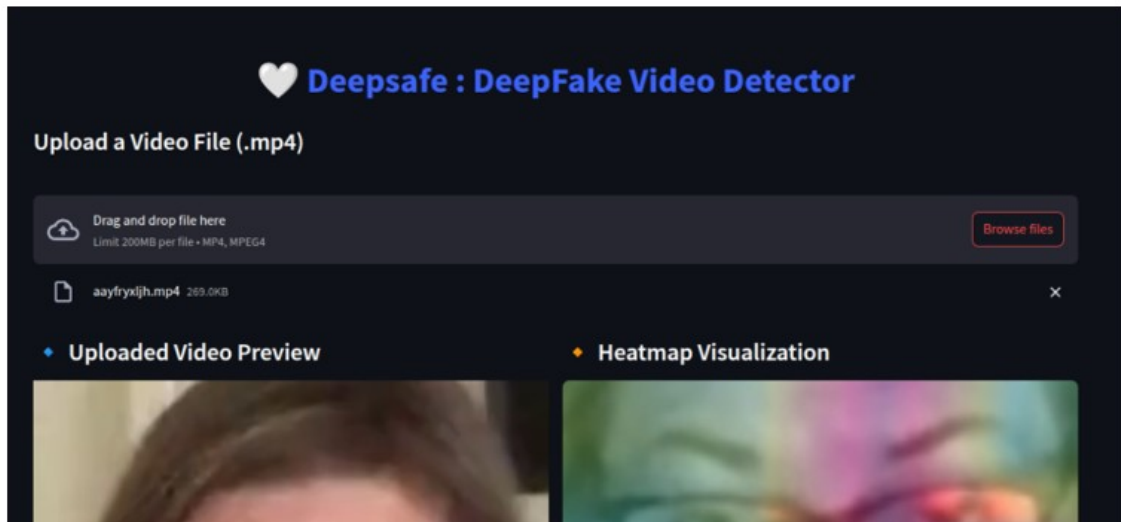


Uploaded real video

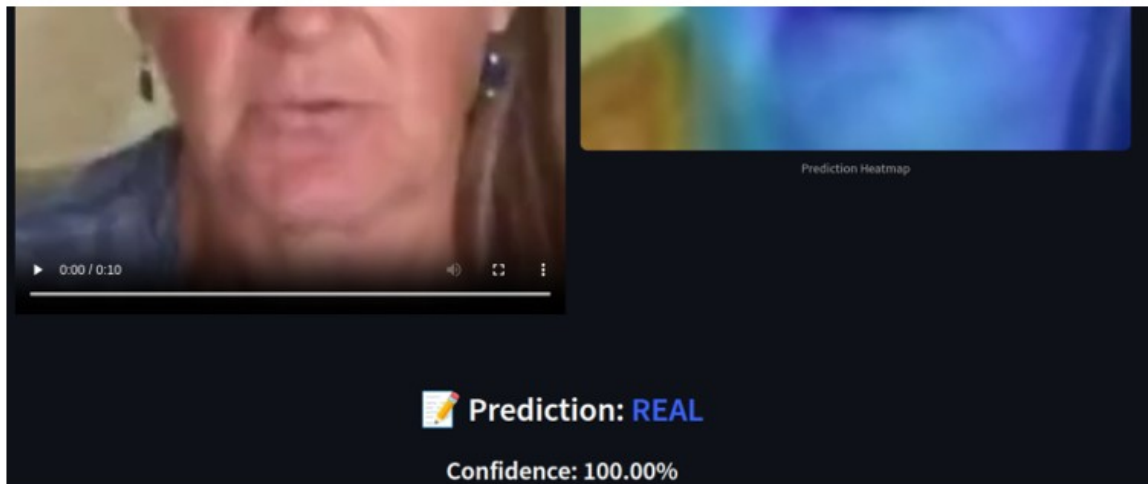


Output as a real video

2.

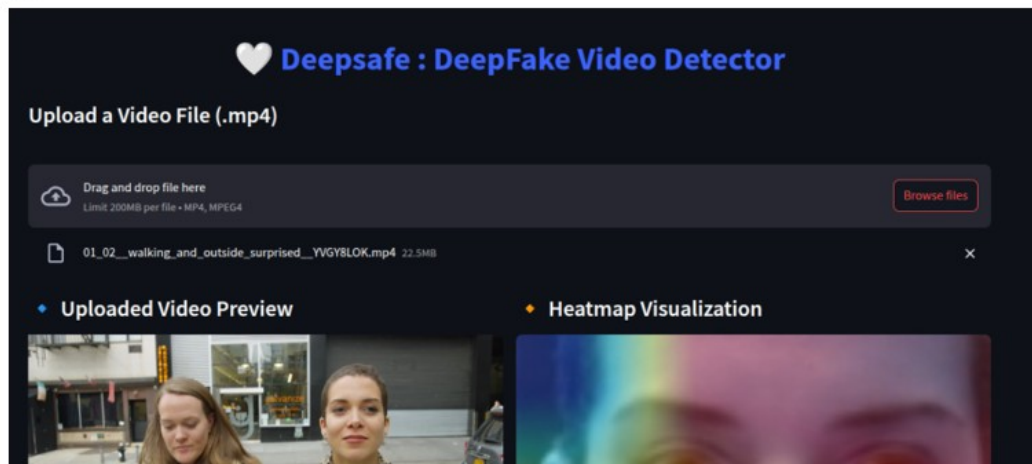


Uploaded a real video

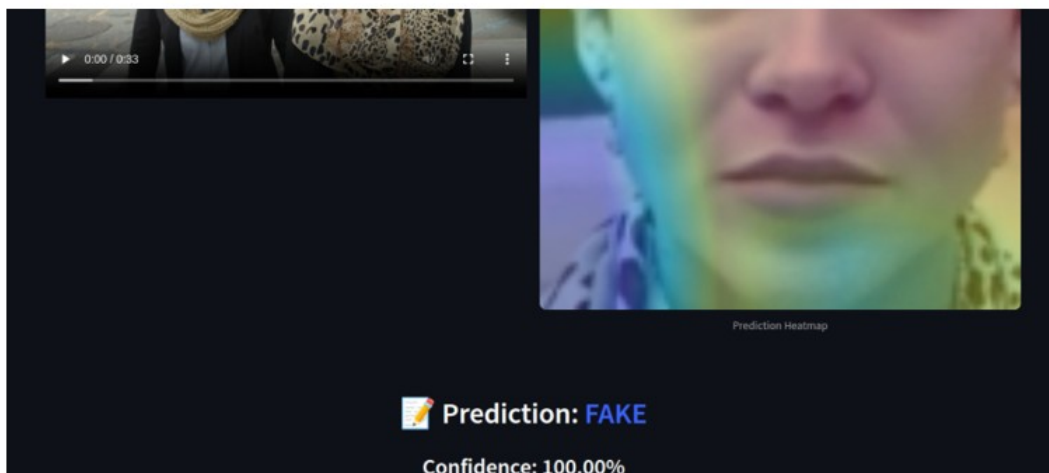


Output as a real video

3.

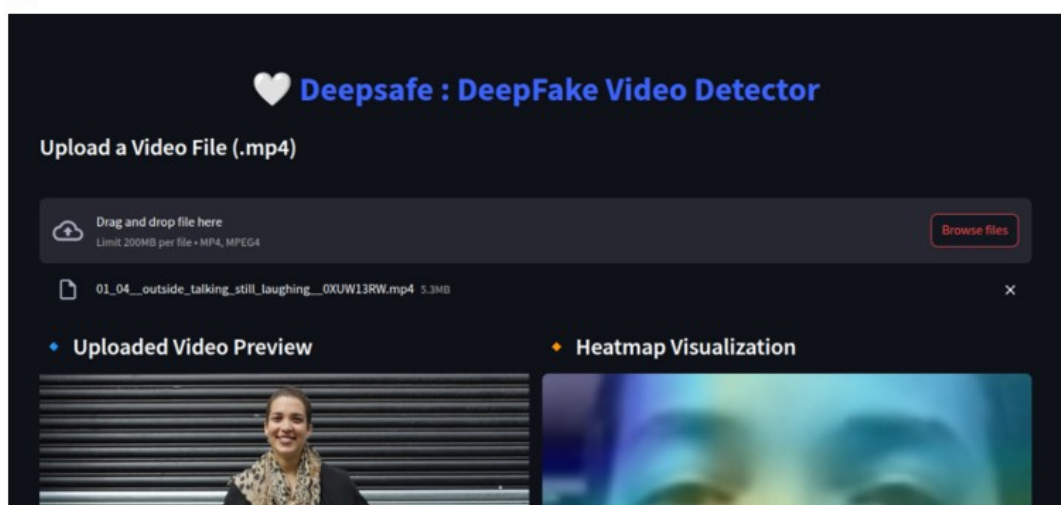


Uploaded a fake video

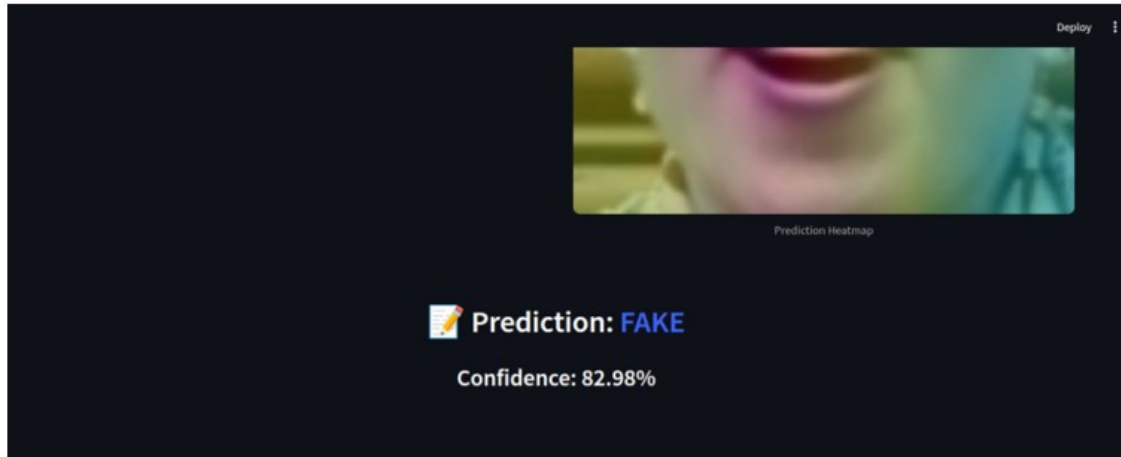


Output as a fake video

4.



Uploaded a fake video



Output as a fake video

10.2 Outputs

10.2.1 Model Results

Model Name	Dataset	No. of videos	Sequence length	Accuracy
model_90_acc_20_frames_FF_data	FaceForensic++	2000	20	90.95477
model_95_acc_40_frames_FF_data	FaceForensic++	2000	40	95.22613
model_97_acc_60_frames_FF_data	FaceForensic++	2000	60	97.48743
model_97_acc_80_frames_FF_data	FaceForensic++	2000	80	97.73366
model_97_acc_100_frames_FF_data	FaceForensic++	2000	100	97.76180
model_93_acc_100_frames_celeb_FF_data	Celeb-DF + FaceForensic++	3000	100	93.97781
model_87_acc_20_frames_final_data	Our Dataset	6000	20	87.79160
model_84_acc_10_frames_final_data	Our Dataset	6000	10	84.21461
model_89_acc_40_frames_final_data	Our Dataset	6000	40	89.34681

Chapter 11

Launch and System Servicing

11.1 Launch

To deploy the deepfake detection Django application, follow the steps below:

1. **Clone the Repository**
2. **Install PyTorch and Torchvision**
3. **Install Project Dependencies**
Install all necessary Python packages listed in the requirements file:
4. **Run Database Migrations**
Apply database migrations required for the Django backend:
5. **Load Trained Models**
Copy all trained model files into the project's models directory. These models will be used during real-time prediction.
6. **Start the Server**
Launch the Django development server using:

Once the server is running, the application will be accessible via your local IP at port **8000**.

11.2 System Servicing

To update the application with the latest codebase and dependencies, follow the steps below:

1. **Stop the Running Server**
Press “Ctrl + C in the terminal” where the production server is going on to stop it safely.
2. **Pull the Latest Code**
Use the following Git command to fetch the latest version of the application:
3. **Install Updated Dependencies**
Reinstall all updated packages from the requirements file:
4. **Apply Database Migrations**
Ensure that any new database changes are applied:
5. **Update Trained Models (If Applicable)**
If any new trained models have been added, copy them to the model's directory.
6. **Restart the Server**

Chapter 12

Conclusion and Future Scope

12.1 Conclusion

In this project, we presented a neural network-driven method to classify videos as either deepfake or real, along with offering a reliability score for each prediction. Our system is capable of making reliable predictions by processing just 1 second of video at “10 frames per second”, achieving high accuracy.

The model was built using a “pre-trained ResNeXt CNN” to derive detailed frame-wise information and an “LSTM” network to capture time-based dependencies between sequential frames, specifically analysing changes between consecutive frames (at times “t and t-1”).

The architecture can flexibly handle video sequences of varying lengths, including 10, 20, 40, 60, 80, and 100 frames, providing scalability based on input size and computational resources.

12.2 Future Scope

- **Browser Extension Development**
The current web-deployed interface can be further transformed into a browser extension, allowing users to easily check videos for deepfake content directly from their web browsers, increasing accessibility and convenience.
- **Extension to Full Body Deepfakes**
Currently, the model focuses primarily on detecting face-based deepfakes. Going forward, the system could be modified to recognize full body deepfakes, enabling broader coverage and stronger protection against more sophisticated synthetic media attacks.

As the digital ecosystem evolves, expanding the system's functionalities will help in keeping pace with emerging deepfake threats and maintaining the relevance and effectiveness of the detection platform.

Appendix A

References

1. Korshunov, P., & Marcel, S. (2019). Deepfakes: A New Threat to Face Recognition? Assessment and Detection. *IEEE International Conference on Biometrics Theory, Applications, and Systems (BTAS)*.
2. Tolosana, R., Vera-Rodriguez, R., Fierrez, J., Morales, A., & Ortega-Garcia, J. (2020). Deepfakes and Beyond: A Survey of Face Manipulation and Fake Detection. *Information Fusion*, 64, 131-148.
3. Li, Y., Chang, M. C., & Lyu, S. (2020). Exposing DeepFake Videos by Detecting Face Warping Artifacts. *IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*.
4. Agarwal, S., Farid, H., Gu, Y., He, M., Nagano, K., & Li, H. (2019). Protecting World Leaders Against Deep Fakes. *IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*

Appendix B

Project Planner

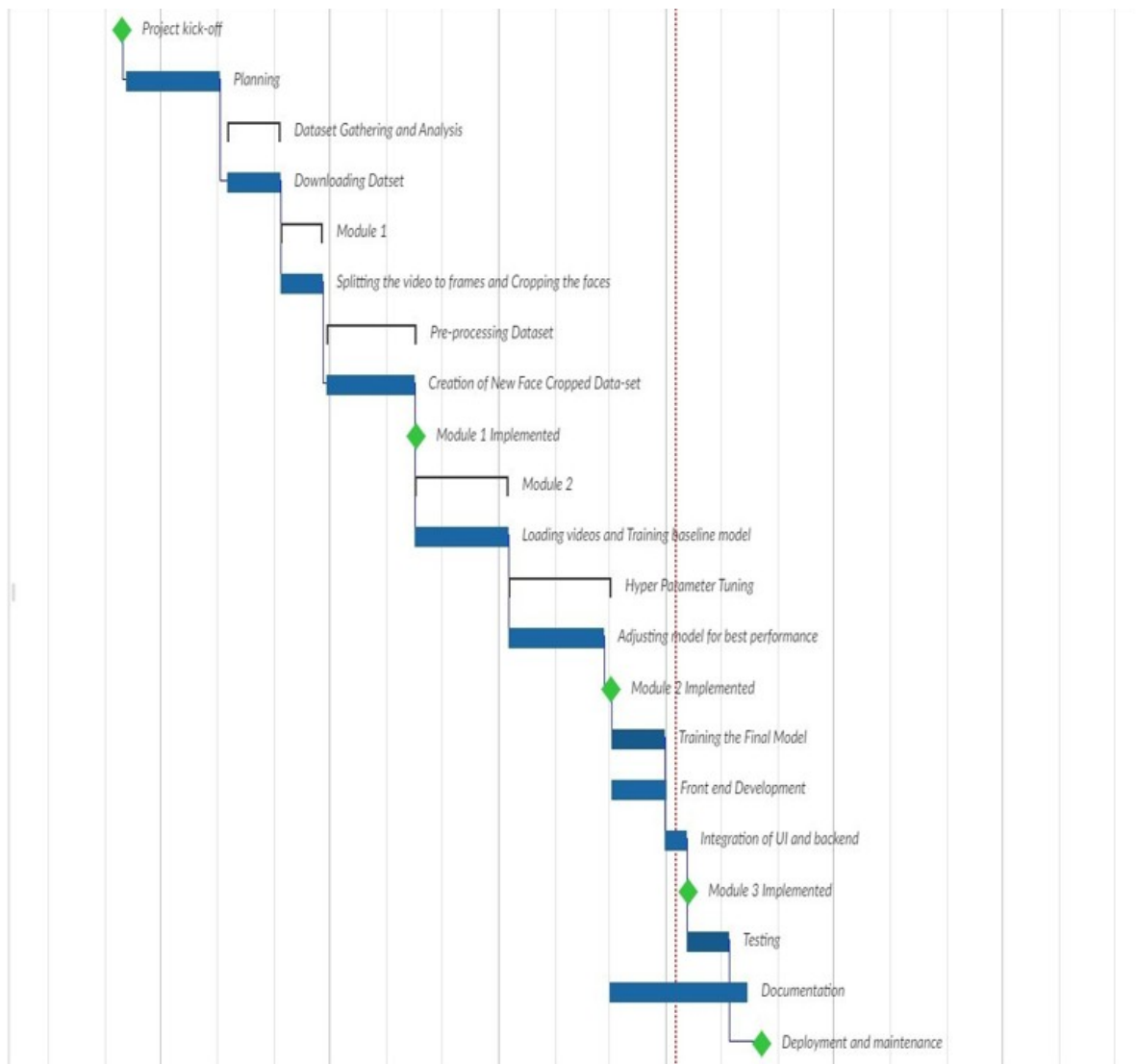


Figure B.1: Complete Project Plan

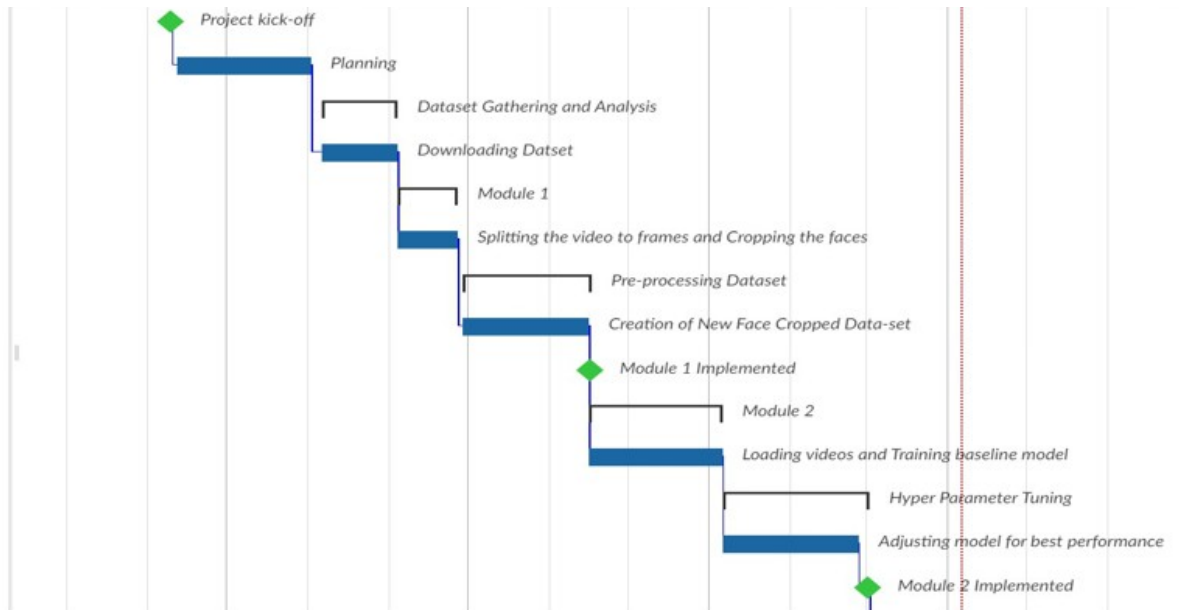


Figure B.2: Project Plan 1

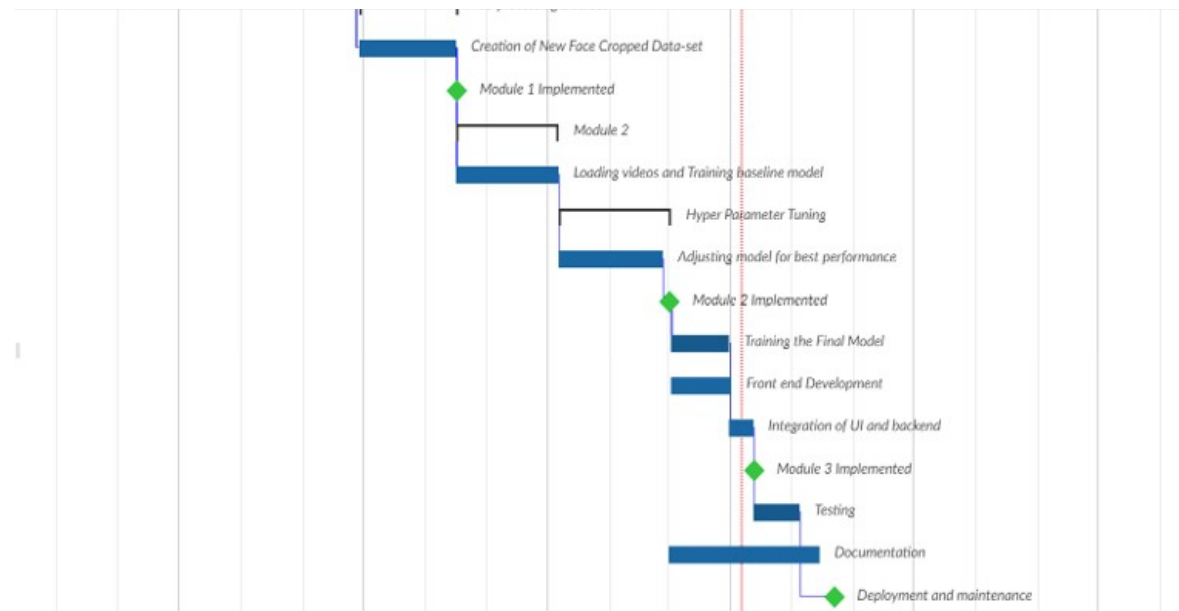


Figure B.3: Project Plan 2